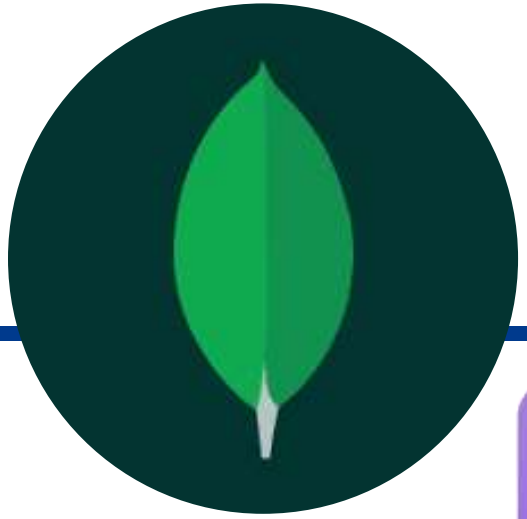
MONGODB, C# .NET







Anas Ben Yaiche

Droits réservés à Cap Compétences, ne pas reproduire

Objectifs de la formation

À la fin de la formation, vous saurez :

1. **Évaluer**
   les solutions NoSQL
2. **Comparer**
   un SGBD relationnel et MongoDB
3. **Configurer**
   un serveur MongoDB
4. **Installer**
   le driver MongoDB pour .NET
5. **Manipuler**
   objets et données en C#
6. **Implémenter**
   une application complète
7. **Écrire**
   des requêtes avec le pipeline d'agrégation

Prérequis

Bonnes connaissances en programmation C#

Avant de commencer

Faisons connaissance, puis testons nos idées reçues sur MongoDB.

Tour de table

- ? Quelles bases de données utilisez-vous aujourd'hui ?
- ? Avez-vous déjà manipulé du JSON ou une base NoSQL ?
- ? Quel problème aimeriez-vous résoudre avec MongoDB ?

Vrai ou faux ?

- V/F *« Sans schéma, pas besoin de modéliser ses données. »*
- V/F *« MongoDB ne gère pas les transactions. »*
- V/F *« Un document MongoDB ne peut pas dépasser 16 Mo. »*

Réponses à la fin du cours.

Sommaire



Introduction à MongoDB



Connexion et utilisation de MongoDB



Intégration de MongoDB avec .NET



Modélisation et indexation



Introduction à la réplication et au Sharding



Extension de MongoDB



Requêtes avancées et agrégations



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

INTRODUCTION À MONGODB

Module 1

Introduction à MongoDB

1. NoSQL : solutions et caractéristiques
2. Introduction au SGBD NoSQL MongoDB
3. La modélisation sous MongoDB
4. Les différents formats utilisés par MongoDB : JSON, BSON
5. Installation de MongoDB
6. Les outils MongoDB



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

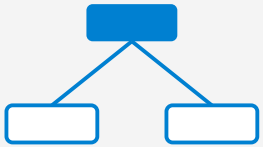
NoSQL : solutions et caractéristiques

Chapitre 1

Historique des bases de données

Chaque modèle de données est né **des limites du précédent**.

1960s

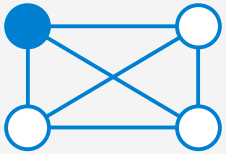


Hiérarchique

IMS (IBM)

Données organisées en arborescence (parent → enfant).

1970s

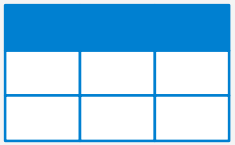


Réseau

CODASYL DBTG

Données reliées par un graphe : relations multiples.

1980s



Relationnel

Oracle, MySQL, PostgreSQL

Tables (lignes / colonnes) manipulées avec SQL.

2000s



NoSQL

BigTable, Cassandra, MongoDB

Schéma flexible et scalabilité horizontale pour le web.

Le mouvement NoSQL

Le NoSQL est né des **besoins de scalabilité des géants du web**.

Fin 1990s

Le relationnel domine

Les SGBD relationnels sont les plus répandus.

2000s

Le mur de la scalabilité

Les géants du web atteignent les limites des SGBD relationnels.

2004–2008

Solutions maison

Google crée BigTable, Facebook crée Cassandra…

Fin 2000s

L'open source s'en mêle

Plusieurs projets open source répondent à ces besoins.

2009

Le terme « NoSQL »

Titre d'une rencontre d'informaticiens à San Francisco.

NoSQL = **« Not Only SQL »** : un complément au relationnel, pas son remplaçant.

Définition

NoSQL = « Not Only SQL »

Des bases qui s'écartent du modèle relationnel, conçues pour **de grands volumes de données peu structurées**.

Schéma flexible

Pas de schéma imposé : les documents d'une même collection peuvent différer.

Scalabilité horizontale

On ajoute des serveurs plutôt que d'acheter une machine plus puissante.

Hautes performances

Lectures et écritures rapides sur de gros volumes, sans jointures coûteuses.

Modèles variés

Document, clé-valeur, colonnes, graphe : un modèle par besoin.

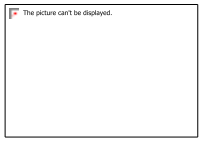
Les 4 familles NoSQL

Quatre façons d'organiser les données, chacune adaptée à un usage.

Clé-valeur Redis, DynamoDB
Simple et rapide : une clé donne une valeur.



Colonne Cassandra, HBase
Orientée Big Data : données rangées par colonnes.



Graphe Neo4j, OrientDB
Relations complexes entre éléments.



Document MongoDB, CouchDB
Flexible : documents JSON / BSON.



Les 4 familles de bases de données NoSQL

Chaque famille stocke les données différemment : le choix dépend de **l'usage**.

Famille	Structure de stockage	Cas d'usage typiques	Exemples
Clé-valeur	Paires clé / valeur simples	Cache, sessions, profils utilisateur	Redis, Riak, DynamoDB
Colonne	Colonnes au lieu de lignes	Big Data, séries temporelles, analytics	Cassandra, HBase, ScyllaDB
Document	Documents JSON / BSON semi-structurés	Applications web, CMS, IoT, API	**MongoDB**, CouchDB, Firestore
Graphe	Nœuds et relations	Réseaux sociaux, recommandations, itinéraires	Neo4j, ArangoDB, JanusGraph

MongoDB appartient à la famille **Document** : c'est celle étudiée dans ce cours.

Le théorème CAP

Théorème CAP

Dans un système distribué, il est impossible de garantir simultanément les 3 propriétés suivantes.

Au maximum, on ne peut en garantir que 2 sur 3 à la fois.



ACID et BASE

ACID bases relationnelles

A — Atomicité
Tout ou rien : la transaction réussit ou échoue entièrement.

C — Cohérence
Les données respectent toujours les règles définies.

I — Isolation
Les transactions concurrentes ne se gênent pas.

D — Durabilité
Une transaction validée survit à un crash.

BASE NoSQL distribuées

BA — Basically Available
Le système répond toujours, quitte à renvoyer des données périmées.

S — Soft State
L'état peut changer sans nouvelle écriture (propagation).

E — Eventually Consistent
Les nœuds finissent par converger vers le même état.

Et MongoDB ? ACID sur un document, et multi-documents depuis la 4.0.

ACID et BASE

Deux philosophies : la **cohérence** d'abord, ou la **disponibilité** d'abord.

Critère	ACID	BASE
Signification	**A · C · I · D** Atomicité, Cohérence, Isolation, Durabilité	**BA · S · E** Basically Available, Soft State, Eventually Consistent
Cohérence	**Stricte et immédiate** chaque lecture voit la dernière écriture	**Éventuelle** les nœuds convergent avec un délai
Transactions	**Complètes** COMMIT / ROLLBACK	**Limitées** MongoDB 4.0+ : multi-documents ACID
Scalabilité	**Verticale (scale-up)** un serveur plus puissant	**Horizontale (scale-out)** on ajoute des nœuds
Disponibilité	**Cohérence d'abord** moins disponible en cas de partition	**Disponibilité d'abord** répond toujours, même avec des données périmées



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Introduction au SGBD NoSQL MongoDB

Chapitre 2

MongoDB

MongoDB est une **base de données orientée document**, open source.

01
Orientée document
Les données sont des documents, regroupés en collections.

02
Open source
Code source public ; éditions Community et Enterprise.

03
Développé en C++
Un moteur compilé, conçu pour la performance.

04
JSON et BSON
On échange en JSON ; MongoDB stocke en BSON (JSON binaire).

05
Schéma flexible
Deux documents d'une même collection peuvent différer.

06
Shell JavaScript
mongosh sert à lire les données et à administrer.

MongoDB : les avantages

Six raisons de choisir MongoDB.

✓

Performance

Conçu dès l'origine pour des lectures et écritures rapides.

✓

Scalabilité native

Réplication et sharding intégrés au moteur.

✓

Requêtes riches

Comparaison, logique, regex, texte, géospatial…

✓

Grands fichiers

GridFS stocke des fichiers volumineux, en grand nombre.

✓

Indexation

Des index sur les documents pour accélérer les requêtes.

✓

Administration simple

Peu de configuration pour démarrer et exploiter.

MongoDB : cas d'utilisation et clients

MongoDB est adapté aux sites web à **fort trafic** et à **forte volumétrie de données**.

Cas d'utilisation

- **Single View**
- **Internet of Things**
- **Mobile**
- **Personalization**
- **Catalog**
- **Realtime Analytics**
- **Content management**
- **Mainframe Offloading**
- **Gaming**

Évolutions clés

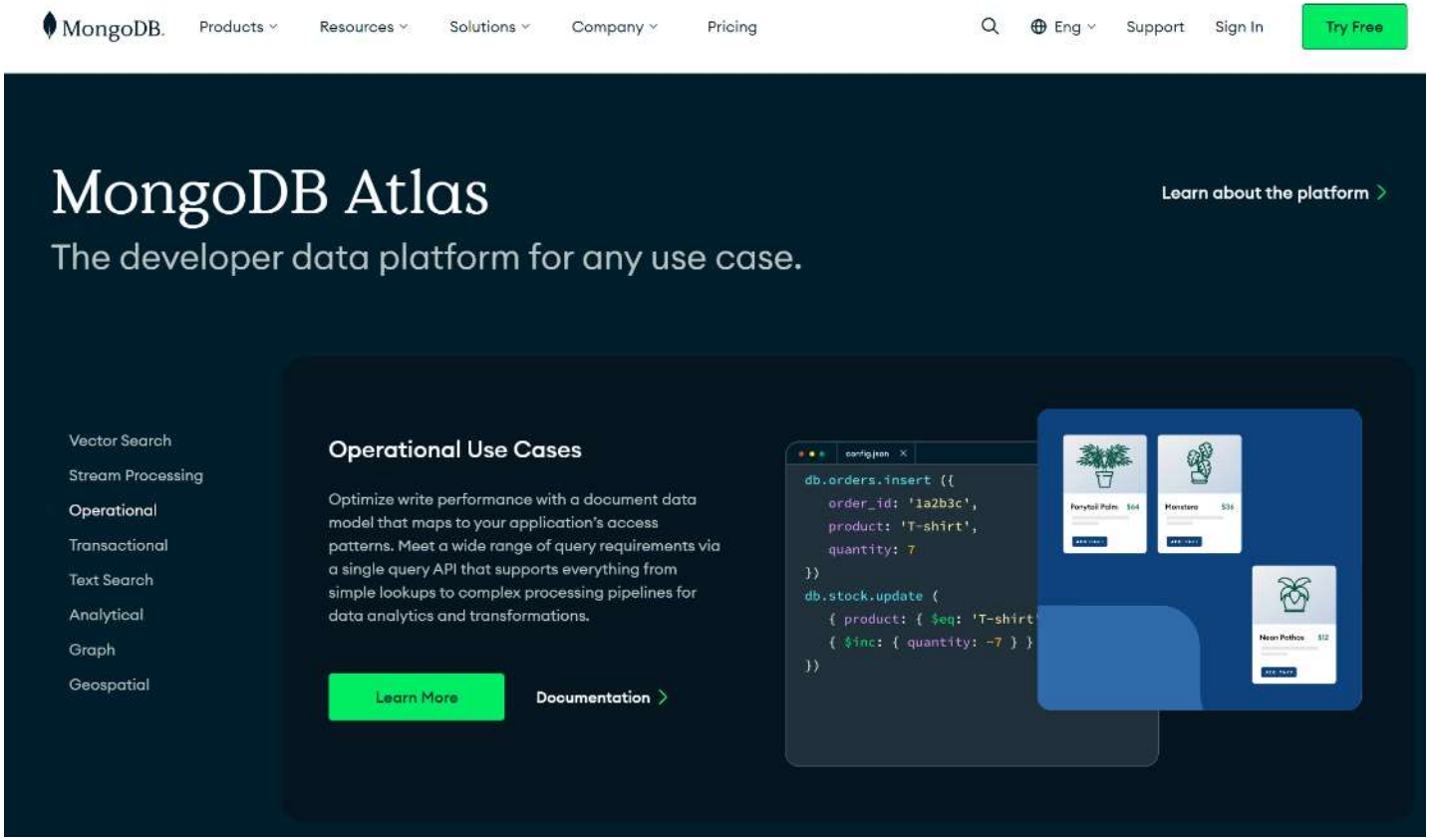
4.0 — **Transactions** (réplication)

4.2 — **Transactions distribuées** (sharding)

5.0 — **Collections** Time Series

5.3 — **Collections** « clustered »

MongoDB Atlas



SQL vs NoSQL


SQL
(Relational Database)
NoSQL
(Not Only SQL)
Modèle de données
Basé sur des tables (lignes et colonnes)
Schéma fixe et prédéfini
Modèle de données
Modèles variés : documents, clés-valeurs, colonnes larges, graphes
Schéma flexible ou dynamique
Exemple de données
ID
Nom
Email
Âge
1
Alice
alice@mail.com
25
2
Bob
bob@mail.com
30
Exemple de données
{
"_id": 1,
"nom": "Alice",
"email": "alice@mail.com",
"age": 25,
"adresses": ["Paris", "Lyon"]
}
Langage de requête
SQL (Structured Query Language)
Langage
Langages de requête spécifiques (souvent inspirés de JSON)
Relations
Relations via clés primaires / étrangères (Jointures)
Relations
Peu ou pas de relations (données souvent dénormalisées)
Transactions
Transactions ACID complètes (Atomicité, Cohérence, Isolation, Durabilité)
Transactions
Transactions ACID partielles (selon le système)
Priorité à la disponibilité et à la performance
Évolutivité
Évolutivité verticale (ajout de ressources sur une machine)
Évolutivité
Évolutivité horizontale (ajout de machines/nœuds facilement)
Cas d'utilisation
Applications ayant des données structurées, transactions complexes et fortes contraintes de cohérence.
Ex : ERP, CRM, systèmes bancaires
Cas d'utilisation
Applications nécessitant une grande scalabilité, des données semi-structurées ou non structurées, et une forte disponibilité.
Ex : Big Data, IoT, réseaux sociaux, catalogues
Exemples de bases
MySQL
PostgreSQL
SQL Server
ORACLE
Exemples
MongoDB
Cassandra
Redis
DynamoDB
CouchDB



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

La modélisation sous MongoDB

Chapitre 3

Structures des données

Les données MongoDB s'organisent en **quatre niveaux imbriqués**.

Serveur MongoDB
plusieurs bases de données

Base de données
plusieurs collections

Collection
documents identifiés par une clé unique

Document
propriétés (champs)

Un champ peut contenir :

Une valeur simple
nombre, chaîne, booléen…

Un sous-document
avec ses propres propriétés

Un tableau
de valeurs ou de sous-documents

```
{
  _id: 1,
  nom: "Ada",                  // simple
  adresse: { ville: "Paris" }, // objet
  tags: ["dev", "data"]        // tableau
}
```

La modélisation sous MongoDB

MongoDB **n'impose pas de schéma fixe** : deux documents d'une collection peuvent différer.

Collection clients

même collection, structures différentes

```
{
  nom: "Ada Martin",
  email: "ada@mail.fr"
}
```

```
{
  nom: "Anas Ben Yaiche",
  email: "anas.benyaiche@example.fr",
  telephone: « 0888888888",
  adresses: [ { ville: "Paris" } ]
}
```

En pratique, **les documents restent proches** : la flexibilité sert à faire évoluer le modèle.

Le défi

Équilibrer les besoins de l'application, les performances du moteur et les modèles d'accès aux données.

La règle

Concevoir selon l'usage réel (requêtes, mises à jour, traitements) et la structure naturelle des données.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Les formats de MongoDB : JSON et BSON

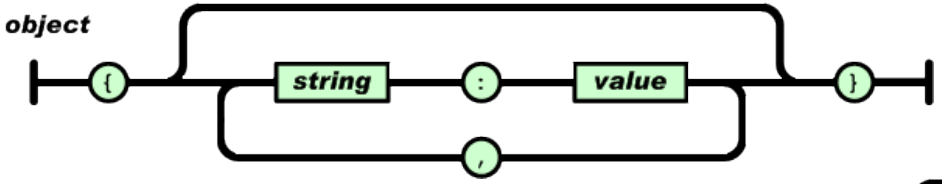
Chapitre 4

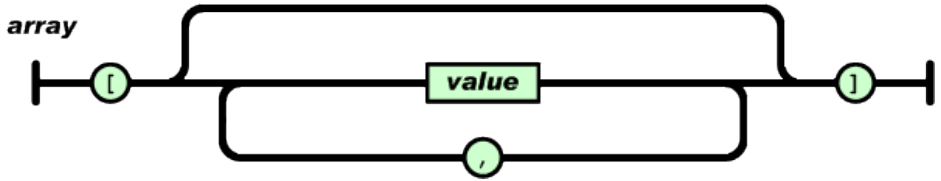
JSON (JavaScript Object Notation)



JSON (JavaScript Object Notation)

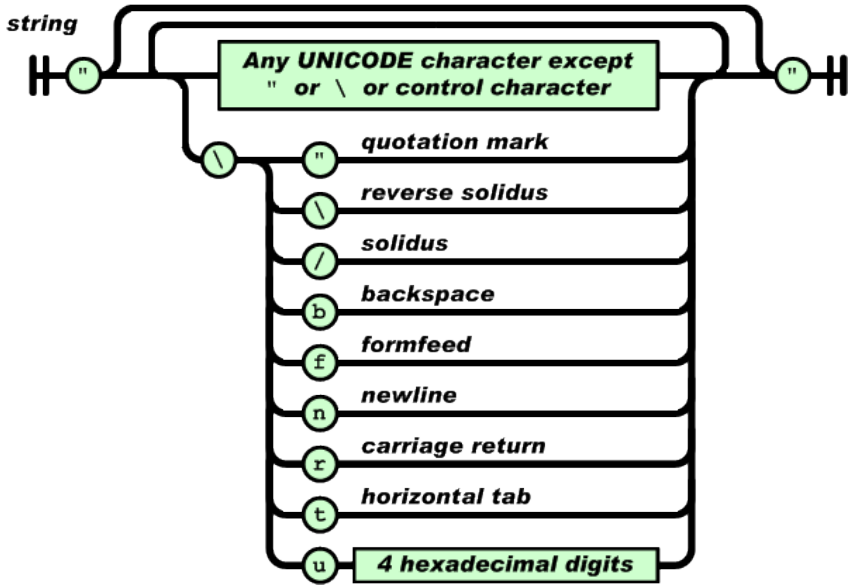
- JSON est un format de données textuelles permettant de représenter de l'information structurée.
- Il est dérivé de la notation des objets en JavaScript
- Crée par Douglas Crockford entre 2002 et 2005, décrit dans la RFC 4627
- Un document JSON comprend deux types d'éléments structurels :

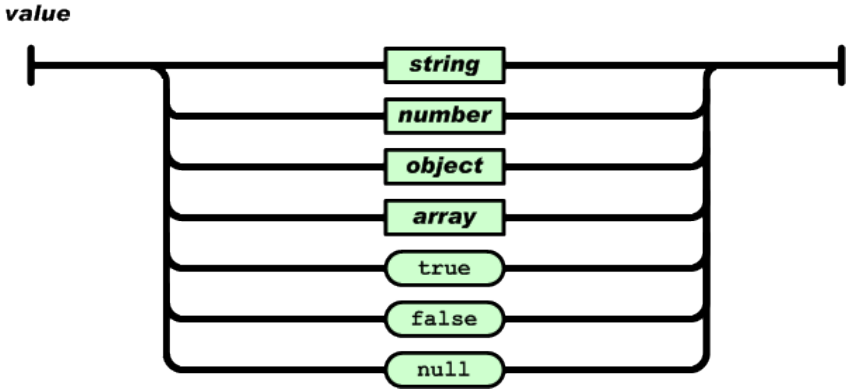


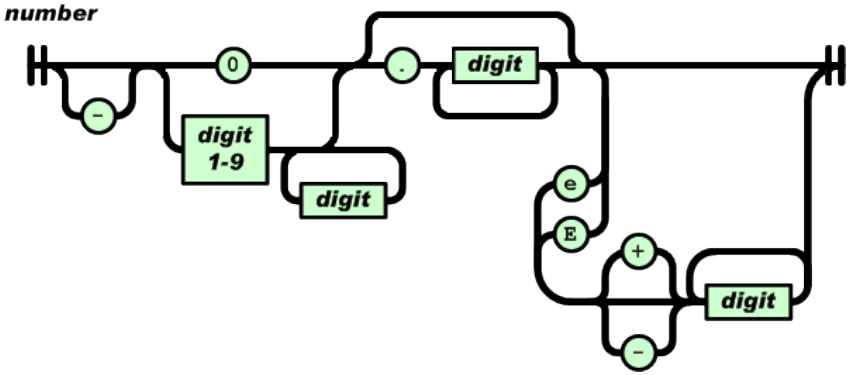


JSON (JavaScript Object Notation)

- Une valeur peut être :







JSON (JavaScript Object Notation)

Pour JavaScript, **un document JSON représente un objet**.

Les avantages du format

Indépendant
des langages de programmation

Simple
types de données faciles à décrire

Lisible
syntaxe claire

Léger
moins volumineux et plus simple que XML

Exemple une voiture

```
{
  "marque": "Citroen",
  "modele": "C4",
  "caracteristiques": {
    "longueur": 3.80,
    "largeur": 1.90,
    "automatique": false
  },
  "couleur": ["blanc", "rouge", "vert", "bleu"]
}
```

BSON (Binary JSON)

BSON est la **représentation binaire** des objets JSON.

JSON
texte lisible

→

BSON
binaire typé

→

MongoDB
stockage et transfert

Types terminaux nombres

Octet 8 bits	**Entier signé** 32 et 64 bits	**Réel** virgule flottante 64 bits	**Décimal** haute précision 128 bits

Autres types

Chaîne UTF-8	**Booléen** true / false	**Date** UTC	**Timestamp** usage interne	**Null**
Tableau […]	**Sous-document** { … }	**Binaire** données brutes	**Regex** expression régulière	**JavaScript** code

Types : identifiants & alias

Type	Num	Alias
Double	1	double
String	2	string
Object	3	object
Array	4	array
Binary data	5	binData
ObjectId	7	objectid
Boolean	8	bool
Date	9	date
Null	10	null
Regular Expression	11	regex
JavaScript	13	javascript
Javascript (w. scope)	15	javascriptWithScope
32-bit integer	16	int
Timestamp	17	timestamp
64-bit integer	18	long
Decimal128	19	decimal

ObjectId

Un ObjectId est une valeur unique de **12 octets**, composée de trois parties dans cet ordre :

4 octets	5 octets	3 octets
Timestamp	**Valeur aléatoire**	**Compteur**
secondes depuis le 1er janvier 1970	propre à la machine et au processus	départ aléatoire

```
ObjectId("6527a1f3a1b2c3d4e50f1a2b")
```

Croissants dans le temps, mais pas strictement ordonnés

- **Résolution d'une seconde** : deux ObjectId créés dans la même seconde n'ont pas d'ordre garanti.
- **Générés par les clients** : les horloges système des machines peuvent différer.

Dates et Timestamps

BSON propose deux types temporels, tous deux stockés sur **64 bits**.

Date le type à utiliser dans les applications

64 bits : entier signé = millisecondes depuis le 1er janvier 1970

- Plage d'environ **290 millions d'années** dans le passé et l'avenir
- Toujours en **UTC**, sans information de fuseau horaire

Timestamp usage interne de MongoDB

32 bits : secondes depuis le 1er janvier 1970	32 bits : compteur d'opérations dans la seconde

- Valeur unique, horodatage du journal des opérations (oplog)…

Dans la plupart des cas, utilisez des **Date** dans vos applications.

MongoDB Extended JSON

JSON ne couvre qu'une partie des types BSON : MongoDB **étend JSON pour conserver les types**.

Mode strict

Conforme à la RFC JSON : l'analyseur interne de MongoDB reconnaît les types.

Mode shell

Fonctions raccourcis gérées par le shell (BinData, ObjectId…).

Exemple : le type « binary »

Strict

```
{ "$binary": "<bindata>", "$type": "<t>" }
```

Shell

```
BinData(<t>, <bindata>)
```

`bindata`	Représentation base64 de la charge binaire
`t`	Un octet qui indique le sous-type de binaire



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Installation de MongoDB

Chapitre 5



mongoDB.

Installation sous Windows

L'installation sous Windows se fait en **quatre étapes**.

1. **Télécharger**
   le programme d'installation sur le site de l'éditeur

```
mongodb.com/download-center
```

2. **Installer**
   MongoDB se place par défaut dans le dossier

```
C:\Program Files\MongoDB\Server\<version>\
```

3. **Configurer (au besoin)**
   le fichier préconfiguré pour un serveur isolé

```
…\Server\<version>\bin\mongod.cfg
```

4. **Démarrer**
   un service Windows est créé et lancé automatiquement

À noter : les outils (mongosh, Compass, Database Tools) s'installent séparément.

Installation sous Linux

La procédure dépend de la distribution, mais **le cheminement reste le même**.

1 Choisir sa distribution

RedHat / CentOS | SUSE | Ubuntu | Debian | Amazon Linux

2 Suivre la procédure officielle

docs.mongodb.com/manual/administration/install-on-linux

```
sudo apt-get install -y mongodb-org     # exemple Ubuntu, après ajout du dépôt MongoDB
```

3 Sans gestionnaire de paquets ?

Installer manuellement depuis une archive **`tarball`** (.tgz), pour les systèmes supportés sans système de paquets.

Installation sous Docker

Trois commandes suffisent pour lancer MongoDB dans un **conteneur Docker** (CentOS).

1 Déclarer le dépôt Docker

```
sudo yum-config-manager --add-repo \
  https://download.docker.com/linux/centos/docker-ce.repo
```

2 Installer Docker

```
sudo yum install docker-ce
```

3 Démarrer MongoDB

```
docker run --name some-mongo -d mongo:tag
```

Lire la commande

`--name`
nom du conteneur

`-d`
lancé en arrière-plan

`mongo:tag`
image et version, ex. mongo:7.0



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Les outils MongoDB

Chapitre 6

mongoDB

Les outils MongoDB

L'installation de MongoDB fournit les exécutables suivants.

SERVEUR
`mongod`
le moteur de base

SERVEUR
`mongos`
le routeur de sharding

SHELL
`mongosh`
le shell interactif

IMPORT / EXPORT
`mongoimport` · `mongoexport`
données JSON, CSV, TSV

SAUVEGARDE
`mongodump` · `mongorestore`
sauvegarde et restauration BSON

CONVERSION
`bsondump`
BSON ➔ JSON

GRIDFS
`mongofiles`
gérer les fichiers stockés

SUPERVISION
`mongostat` · `mongotop`
statut d'une instance, temps de lecture / écriture

ENTERPRISE
`mongoldap` · `mongokerberos`
tester LDAP et Kerberos



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

CONNEXION ET UTILISATION DE MONGODB

MODULE 2

mongo DB

Connexion et utilisation de MongoDB

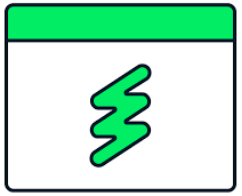
Ce chapitre : **prendre en main MongoDB avec le shell**.

1. **Utiliser le shell Mongo**
   Se connecter et lancer ses premières commandes.

2. **Les objets MongoDB**
   Bases, collections, documents et espaces de noms.

3. **Manipuler les données**
   Les opérations CRUD : créer, lire, modifier, supprimer.

4. **Écrire des scripts**
   Automatiser des tâches avec le shell.





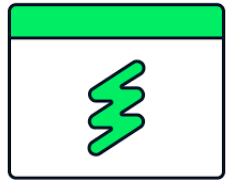


CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Utilisation du shell Mongo

Chapitre 1



Mongosh

`mongosh` est le shell interactif de MongoDB.

Syntaxe l'adresse est une chaîne de connexion

```
mongosh [options] [db address]
mongosh "mongodb://mongodb0.example.com:27017/testdb?tls=true"
```

Quelques options

`--port <port>`	Port différent de 27017
`--host <hostname>`	Hôte différent de localhost
`--file <script.js>`	Exécuter un script JS

Embarque Node.js

Les scripts exécutés dans mongosh profitent de tout l'écosystème Node.js.

Premières commandes d'administration

Premières commandes à taper dans le shell `mongosh`.

Naviguer

Commande	Rôle
`show dbs`	liste les bases de données
`db`	affiche la base en cours
`use <db>`	change de base (« test » par défaut)
`show collections`	liste les collections de la base
`show users`	liste les utilisateurs de la base
`show logs`	liste les logs disponibles
`show log [name]`	affiche un log (« global » par défaut)
`exit`	quitte le shell

Obtenir de l'aide

Commande	Aide sur...
`help`	le shell
`db.help()`	les méthodes de la base
`db.<coll>.help()`	les méthodes d'une collection



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Les bases, les collections, les documents et les espaces de noms

Chapitre 2

Les bases de données

Le nom d'une base de données doit respecter **quelques règles**.

✗ Le nom ne peut pas...

Être vide
ni être le caractère null

Être un nom système
`admin · local · config`

Contenir sous Windows
`/ \ . " $ * < > : | ? espace`

Contenir sous Unix / Linux
`/ \ . " $ espace`

À savoir

64 caractères
longueur maximale du nom

Insensible à la casse
MaBase et mabase ne peuvent pas coexister

Les bases de données

Le cycle de vie d'une base : **elle se crée seule**, mais se supprime explicitement.

1. Créer

La base apparaît à la **première insertion**.

```
mongosh test
// ou : use test

db.clients.insertOne(
  { name: "Anas" })
// → "banque" existe
```

2. Supprimer

Supprime la **base en cours** et ses collections.

```
// commande d'administration
db.runCommand(
  { dropDatabase: 1 })

// raccourci du shell
db.dropDatabase()
```

Attention : les utilisateurs définis dans la base ne sont pas supprimés.
Lancez d'abord `db.dropAllUsersFromDatabase()`.

Les collections

Une collection est **créée automatiquement** à la première insertion d'un document, ou explicitement :

```
db.createCollection(name, [options])
```

Collection « cappée » taille limitée, mode FIFO

`capped`	true pour limiter la taille
`size`	taille maximum, en octets
`max`	nombre maximum de documents

Validation règles sur les documents

`validator`	document de règles
`validationLevel`	`strict` : toutes les écritures `moderate` : sauf documents existants invalides
`validationAction`	`error` : l'écriture échoue `warn` : alerte dans les logs

Exemple

```
db.createCollection("logs", { capped: true, size: 1048576, max: 5000 })
```

Les collections

Nommer une collection

✗ Une chaîne vide

✗ Contenir `null ou $`

✗ Commencer par `system.`

✓ Sous-collections : `Blog.posts`

Lister les collections de la base en cours

```
show collections
db.getCollectionNames()
```

Supprimer une collection de la base en cours

```
db.produits.drop()
```

Les collections « timeseries » (>=5.0)

- Une collection « timeseries » stocke des mesures au cours du temps de manière efficace. Ces mesures étant identifiées par des métadonnées qui change très rarement :

Sujet	Mesure	Métadonnées
Météo	Température	Numéro de capteur, localisation
Bourse	Valeur d'une action	Code de l'action, bourse
Fréquentation de site web	Nombre de vues	URL

- Stockées en interne de manière optimisé, ces collections optimisent les performances des requêtes, l'utilisation du cache correspondant et réduisent le volume de données (stocké, entrant, sortant).

Les collections « timeseries » (>=5.0)

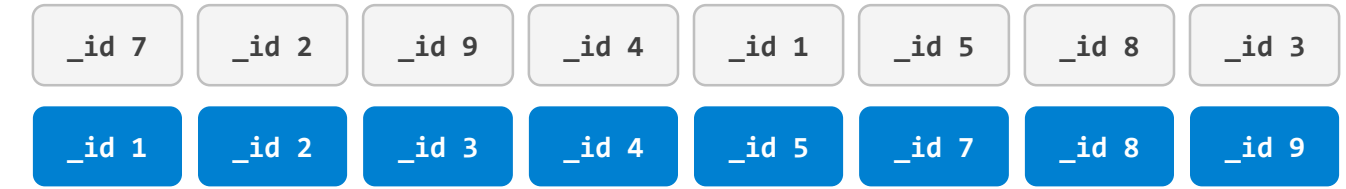
Une collection « timeseries » se crée avec l'option `timeseries` :

```
db.createCollection("mesures", {
  timeseries: {
    timeField: "date",        // obligatoire
    metaField: "capteur",     // optionnel
    granularity: "minutes"    // optionnel
  },
  expireAfterSeconds: 86400   // optionnel
})
```

Option	Rôle
`timeField`	Champ qui contient la date de la mesure **obligatoire**
`metaField`	Champ des métadonnées (qui changent rarement) optionnel
`granularity`	`seconds` (défaut), `minutes` ou `hours` : la plus proche de l'intervalle entre deux mesures
`expireAfterSeconds`	Suppression automatique des mesures au bout de N secondes optionnel, pas d'expiration par défaut

Les collections « clustered » (>=5.3)

Une collection « clustered » stocke les documents **dans l'ordre de la clé de clustering**.



Bénéfices

Requêtes plus rapides
sur un intervalle de valeurs de la clé, ou une égalité

Moins de stockage
et insertions par lot plus performantes

Un index en moins
si le champ recherché est la clé de clustering

TTL intégré
suppressions plus performantes

Un seul index « clustered » par collection, défini uniquement à la création de la collection.

Les collections « clustered » (>=5.3)

Une collection « clustered » se crée avec l'option `clusteredIndex` :

```
db.createCollection("commandes", {
  clusteredIndex: {
    key: { _id: 1 },      // obligatoire
    unique: true,         // obligatoire
    name: "cmd_cluster"   // optionnel
  }
})
```

Inconvénients

Impossible de passer une collection classique en « clustered », et inversement.

Si un autre index existe, il faut préciser que l'on veut utiliser l'index « clustered ».

Une collection « clustered » ne peut pas être « cappée ».

Les collections « clustered » (>=5.3)

- Les clé de custering :
  - doivent contenir des valeurs uniques.
  - doivent contenir des valeurs immuables.
  - devraient contenir des valeurs augmentant de manière séquentielle pour optimiser les insertions.
  - devraient aussi petites que possible pour éviter de pénaliser les performances et d'anéantir les avantages en termes de taille de stockage.

Les documents

Les documents sont exprimés au **format JSON** et insérés dans une collection.

Insertion

```
db.<nom_collection>.insertOne(<document>)

db.clients.insertOne({ nom: "Ada" })
```

Document stocké _id ajouté et indexé

```
{
  _id: ObjectId("65f1c2…"),   // clé primaire
  nom: "Ada"
}
```

Règles sur les clés

✗

Interdit

null, « . » ou « $ » en début de clé

1

Unique

une clé n'apparaît qu'une fois par document

Aa

Sensible à la casse

nom et Nom sont deux clés différentes

Les espaces de noms

Chaque collection et chaque index possède un **espace de noms** : son nom complet.

`<base>`
nom de la base

•

`<collection>`
nom de la collection ou de l'index

Exemples

```
banque.clients
```
→ la collection clients de la base banque

```
banque.transactions
```
→ la collection transactions

```
blog.posts.comments
```
→ une sous-collection (le point est permis)

Limite : 255 octets au maximum pour l'espace de noms complet. Gardez des noms de base et de collection courts.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Manipulation des objets et données : les opérations CRUD

- Chapitre 3

Les instructions CRUD à travers le shell – Create

On ajoute des documents avec **insertOne** ou **insertMany**.

insertOne

```
db.<collection>.insertOne(
  <document>,
  { writeConcern: <document> }
)
```

insertMany

```
db.<collection>.insertMany(
  [ <document>, … ],
  { writeConcern: <document>,
    ordered: <boolean> }
)
```

writeConcern	Stratégie de prise en compte de l'écriture
ordered: true	Insère dans l'ordre et s'arrête à la première erreur (défaut)
ordered: false	Insère dans n'importe quel ordre et continue malgré les erreurs

Exemples

```
db.produits.insertOne({ name: "lecteur DVD", prix: 199.99 })
db.produits.insertMany([{ name: "lecteur DVD", prix: 199.99 }, … ])
```

Les instructions CRUD à travers le shell – Read

On lit des documents avec la méthode **`find`**.

Sans paramètre

Retourne tous les documents de la collection.

```
db.produits.find()
```

Avec un filtre

Un document JSON décrit les conditions que les documents doivent remplir.

```
db.produits.find({ <conditions> })
```

Exemples chaque condition ajoutée affine le résultat

```
db.produits.find({ type: "canapé" })
db.produits.find({ type: "canapé", matiere: "cuir" })
db.produits.find({ type: "canapé", matiere: "cuir", couleur: "noir" })
```

Les instructions CRUD à travers le shell – Read

Les **opérateurs de comparaison** remplacent une valeur fixe dans le filtre.

Opérateur	Signification	Exemple
`$gt`	supérieur à	`{ prix: { $gt: 100 } }`
`$gte`	supérieur ou égal à	`{ prix: { $gte: 100 } }`
`$lt`	inférieur à	`{ prix: { $lt: 600 } }`
`$lte`	inférieur ou égal à	`{ prix: { $lte: 600 } }`
`$ne`	différent de	`{ type: { $ne: "canapé" } }`
`$in`	compris dans la liste	`{ couleur: { $in: ["noir", "blanc"] } }`
`$nin`	non compris dans la liste	`{ couleur: { $nin: ["rouge"] } }`

Exemples

```
db.produits.find({ type: "canapé", prix: { $lt: 600 } });
db.produits.find({ couleur: { $in: ["noir", "blanc"] } });
```

Les instructions CRUD à travers le shell – Read

Les **opérateurs logiques** combinent plusieurs conditions.

Opérateur	Signification	Syntaxe
`$and`	ET : toutes vraies	`{ $and: [ { <expr1> }, { <expr2> } ] }`
`$or`	OU : au moins une vraie	`{ $or: [ { <expr1> }, { <expr2> } ] }`
`$not`	NON : inverse l'expression	`{ champ: { $not: { <expr> } } }`
`$nor`	NI : toutes fausses	`{ $nor: [ { <expr1> }, { <expr2> } ] }`

Exemple produits entre 600 et 900

```
db.produits.find({ $and: [
  { prix: { $gte: 600 } },
  { prix: { $lt: 900 } }
] });
```

Astuce : sur des champs différents, un $and implicite suffit : `{ type: "canapé", prix: { $lt: 900 } }`

Les instructions CRUD à travers le shell – Read

D'autres opérateurs testent **l'existence, le type** d'un champ ou le contenu **d'un tableau**.

Opérateur	Le champ doit…	Exemple
Éléments		
`$exists`	exister (true) ou non (false)	`{ email: { $exists: true } }`
`$type`	être du type indiqué (alias ou numéro)	`{ prix: { $type: "double" } }`
Tableaux		
`$all`	contenir toutes les valeurs	`{ tags: { $all: ["bio", "local"] } }`
`$elemMatch`	avoir au moins un élément qui vérifie tous les critères	`{ results: { $elemMatch: { $gte: 14 } } }`
`$size`	contenir exactement n éléments	`{ tags: { $size: 3 } }`

Exemple au moins une note entre 14 (inclus) et 16 (exclu)

```
db.scores.find({ results: { $elemMatch: { $gte: 14, $lt: 16 } } })
```

Les instructions CRUD à travers le shell – Read

- $expr (≥ 3.6) : des opérateurs d'agrégation dans un filtre, ex. comparer deux champs.

Collection budgetMensuel 5 documents

```
{ _id: 1, categorie: "alimentaires", budget: 400, depense: 450 }
{ _id: 2, categorie: "boissons",     budget: 100, depense: 150 }
{ _id: 3, categorie: "vetements",    budget: 100, depense: 50 }
{ _id: 4, categorie: "autres",       budget: 500, depense: 300 }
{ _id: 5, categorie: "voyages",      budget: 200, depense: 650 }
```

Requête documents dont la dépense dépasse le budget

```
db.budgetMensuel.find({ $expr: { $gt: ["$depense", "$budget"] } })
```

Résultat 3 documents sur 5

```
{ _id: 1, categorie: "alimentaires", budget: 400, depense: 450 }
{ _id: 2, categorie: "boissons",     budget: 100, depense: 150 }
{ _id: 5, categorie: "voyages",      budget: 200, depense: 650 }
```

Les instructions CRUD à travers le shell – Read

Une **projection** restreint les champs renvoyés pour chaque document.

```
db.<collection>.find(<query>, { field1: <value>, field2: <value>, … })
```

Inclure un champ

valeur `1 ou true`

```
db.produits.find({}, { name: 1, prix: 1 })
```

Exclure un champ

valeur `0 ou false`

```
db.produits.find({}, { description: 0 })
```

Pas de mélange

On ne peut pas inclure et exclure des champs dans la même projection.

Le champ _id

Il est inclus par défaut, sauf si on l'exclut explicitement : { _id: 0 }.

Les instructions CRUD à travers le shell – Curseur et boucle

`find()` renvoie un **curseur** : il faut le parcourir pour accéder aux documents.

> **Curseur non affecté à une variable :** le shell l'itère tout seul et affiche les **20 premiers** résultats.

Boucle while hasNext() / next()

```
var monCurseur = db.stocks.find();
while (monCurseur.hasNext()) {
  printjson(monCurseur.next());
}
```

forEach() une fonction par document

```
db.contacts.find().forEach(
  function (doc) {
    print(doc.nom);
  });
```

Comme un tableau toArray() charge tout en mémoire

```
var monCurseur = db.stocks.find();
var docArray = monCurseur.toArray();
print(docArray[0].nom);
```

Les instructions CRUD à travers le shell – Tri

`cursor.sort()` trie les résultats ; on l'applique **avant de récupérer les documents**.

Sur un ou plusieurs champs `1` croissant · `-1` décroissant

```
monCurseur.sort({ age: 1 })
monCurseur.sort({ age: 1, note: -1 })   // âge croissant, puis note décroissante
```

Sans index, **le tri en mémoire est limité à 100 Mo** : au-delà, MongoDB renvoie une erreur.

1. Créer un index

MongoDB trie alors grâce à l'index.

```
db.eleves.createIndex({ age: 1 })
```

2. Limiter le résultat

Moins de documents à trier.

```
monCurseur.sort({ age: 1 })
  .limit(10)
```

3. Autoriser le disque

Fichiers temporaires sur disque.

```
monCurseur.sort({ age: 1 })
  .allowDiskUse()
```

Les instructions CRUD à travers le shell – Collation

`cursor.collation()` trie et compare les chaînes **selon les règles d'une langue**.

Syntaxe seule la langue (locale) est obligatoire

```
monCurseur.collation({ locale: "fr" })
```

Pourquoi ? Tri de noms avec `sort({ nom: 1 })`

Sans collation

ordre binaire : majuscules, puis accents à la fin

```
Zoé
alice
éric
```

Avec { locale: "fr" }

ordre alphabétique français

```
alice
éric
Zoé
```

Les autres paramètres (casse, accents…) prennent **les valeurs adaptées à la langue** choisie. Détails : docs.mongodb.com/manual/reference/collation

Les instructions CRUD à travers le shell – findOne

`findOne()` renvoie **un seul document** (ou null), et non un curseur.

```
db.<collection>.findOne(<query>, <projection>)
```

`<query>`	filtre de recherche · *optionnel*	`<projection>`	champs à renvoyer · *optionnel*

Exemple

```
db.clients.findOne(
  { ville: "Paris" },
  { nom: 1, email: 1, _id: 0 })
```

Résultat

```
{ nom: "Anas Ben Yaiche",
  email: "anas.benyaiche@mail.fr" }
```

Pas de tri possible

Le document renvoyé est le premier trouvé dans l'ordre naturel (souvent l'ordre d'insertion, sans garantie). Pour choisir lequel : **`find().sort(…).limit(1)`**

Les instructions CRUD à travers le shell – Update

Trois fonctions modifient le contenu d'une collection.

`updateOne()`
modifie le premier document trouvé

`updateMany()`
modifie tous les documents trouvés

`replaceOne()`
remplace tout le document (sauf _id)

Syntaxe identique pour les trois

```
db.<collection>.updateOne(<query>, <update>, {
  upsert: <booléen>,
  writeConcern: <document>
})
```

Paramètre	Rôle
`query`	critères de sélection des documents à modifier
`update`	modifications à apporter (ou document de remplacement)
`upsert`	si vrai, insère un document quand aucun ne correspond (défaut : faux)
`writeConcern`	stratégie de prise en compte de l'écriture

Les instructions CRUD à travers le shell – Update

Les **opérateurs de mise à jour** modifient des champs précis sans réécrire le document.

Opérateur	Effet	Exemple
`$set`	modifie ou ajoute des champs	`{ $set: { marque: "Sony" } }`
`$inc`	incrémente d'une valeur	`{ $inc: { stock: -1 } }`
`$mul`	multiplie par une valeur	`{ $mul: { prix: 1.1 } }`
`$rename`	renomme des champs	`{ $rename: { nom: "name" } }`
`$unset`	supprime des champs	`{ $unset: { promo: "" } }`
`$currentDate`	insère la date courante	`{ $currentDate: { modifie: true } }`

Il en existe d'autres, notamment pour les tableaux : `$push`, `$pull`, `$addToSet`…

Les instructions CRUD à travers le shell – Update

Trois exemples, du remplacement complet à la mise à jour ciblée.

replaceOne remplace tout le document

```
db.produits.replaceOne({ _id: "123456789" }, { marque: "Sony", modele: "Xperia" })
```

updateMany modifie tous les documents filtrés

```
db.produits.updateMany({ marque: "Sony Ericsson" }, { $set: { marque: "Sony" } })
```

updateOne plusieurs opérateurs dans une seule mise à jour

```
db.users.updateOne({ _id: 1 }, {
  $currentDate: {
    lastModified: { $type: "date" },              // => Date UTC
    "cancellation.date": { $type: "timestamp" }   // => Timestamp
  },
  $set: { status: "D", "cancellation.reason": "user request" }
})
```

Les instructions CRUD à travers le shell – Update

Le 2e paramètre d'une mise à jour peut être **un pipeline d'agrégation** (≥ 4.2).

```
db.collection.updateMany(
  <query>,
  [
    { $set: { status: "Modified",
              comments: ["$misc1", "$misc2"] } },
    { $unset: ["misc1", "misc2"] }
  ]
)
```

1. **`$set`**
   ajoute status et regroupe misc1 et misc2 dans comments

2. **`$unset`**
   supprime ensuite misc1 et misc2

Étapes autorisées dans ce pipeline

`$addFields` / `$set`	`$project` / `$unset`	`$replaceRoot` / `$replaceWith`

Les instructions CRUD à travers le shell – findOneAndReplace

`findOneAndReplace` cherche un document, le remplace et le retourne.

```
db.<collection>.findOneAndReplace(
  <filter>, <replacement>, {
  sort: <document>,
  projection: <document>,
  upsert: <boolean>,
  returnNewDocument: <boolean>,
  collation: <document>
})
```

Paramètre	Rôle
`filter`	filtre de recherche
`replacement`	document de remplacement
`sort`	ordre de tri si plusieurs documents correspondent
`projection`	champs du document retourné
`upsert`	insère un document si aucun ne correspond (défaut : faux)
`returnNewDocument`	retourne le nouveau document plutôt que l'ancien (défaut : faux)
`collation`	règles de comparaison selon une langue

Les instructions CRUD à travers le shell – findOneAndUpdate

`findOneAndUpdate` cherche un document, le modifie et le retourne.

```
db.<collection>.findOneAndUpdate(
  <filter>, <update>, {
  sort: <document>,
  projection: <document>,
  upsert: <boolean>,
  returnNewDocument: <boolean>,
  collation: <document>,
  arrayFilters: [ <filtre>, … ]
})
```

Paramètre	Rôle
`filter`	filtre de recherche
`update`	modifications (ou pipeline d'agrégation, ≥ 4.2)
`sort`	ordre de tri si plusieurs documents correspondent
`projection`	champs du document retourné
`upsert`	insère un document si aucun ne correspond (défaut : faux)
`returnNewDocument`	retourne le document modifié plutôt que l'original (défaut : faux)
`collation`	règles de comparaison selon une langue
`arrayFilters`	éléments de tableau à modifier

Les instructions CRUD à travers le shell – Delete

Deux fonctions suppriment des documents : **un seul** ou **tous ceux qui correspondent** au filtre.

`deleteOne` le 1er document qui correspond

```
db.<collection>.deleteOne(
  <query>, { writeConcern: <document> })
```

`deleteMany` tous les documents qui correspondent

```
db.<collection>.deleteMany(
  <query>, { writeConcern: <document> })
```

`<query>`	critères de sélection des documents à supprimer
`writeConcern`	stratégie de prise en compte de l'écriture · optionnel

Exemples

```
db.stocks.deleteOne({ code: "123456789" })
db.stocks.deleteMany({ type: "périssable" })
db.stocks.deleteMany({})   // vide la collection
```

Filtre vide { }
supprime tous les documents, sans retour arrière.

Les instructions CRUD à travers le shell – findOneAndDelete

`findOneAndDelete()` cherche un document, le supprime et **renvoie le document supprimé**.

Signature

```
db.<collection>.findOneAndDelete(
  <filter>,
  { sort: <document>,
    projection: <document>,
    collation: <document> }
)
```

Paramètre	Rôle
`filter`	Critères de recherche
`sort`	Lequel supprimer s'il y en a plusieurs
`projection`	Champs du document renvoyé
`collation`	Comparaison selon la langue

Exemple supprimer la plus ancienne transaction rejetée

```
db.transactions.findOneAndDelete(
  { Status: "Rejetée" },
  { sort: { Date: 1 }, projection: { _id: 1, Amount: 1 } }
)
```

Différence avec deleteOne() : on récupère le document, par exemple pour l'archiver ou l'afficher.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Ecrire des scripts pour le shell Mongo

Chapitre 4

Ecrire des scripts pour le shell Mongo

Un script JavaScript s'exécute en **quatre étapes**.

1. Lancer le script depuis le terminal

Avec le client mongosh.

```
mongosh --file myScript.js
```

2. Ou le charger depuis le shell

Dans une session mongosh ouverte.

```
load("myScript.js")
```

3. Se connecter

Dans le script, avec le constructeur Mongo.

```
conn = new Mongo()
conn = new Mongo(<host>)
conn = new Mongo(<host:port>)
```

4. Choisir la base

Via la connexion, ou directement avec connect().

```
db = conn.getDB("myDatabase");
// ou en une seule ligne :
db = connect("localhost:27020/myDatabase");
```

Ecrire des scripts pour le shell Mongo

Dans un script, les aides du shell ne fonctionnent pas : **seul le JavaScript valide** est accepté.

Aide du shell	Équivalent JavaScript
`show dbs`	`db.adminCommand({ listDatabases: 1 })`
`use <db>`	`db = db.getSiblingDB('<db>')`
`show collections`	`db.getCollectionNames()`
`show logs`	`db.adminCommand({ getLog: '*' })`
`show log <logname>`	`db.adminCommand({ getLog: '<logname>' })`

Exemple parcourir un curseur, afficher avec print et printjson

```
conn = new Mongo("localhost.localdomain");
db = conn.getDB("local");
print("switch to local");
cursor = db.startup_log.find();
while (cursor.hasNext()) { printjson(cursor.next()); }
```

.mongoshrc.js

Au lancement, le shell exécute le fichier `.mongoshrc.js` du dossier utilisateur, s'il existe.

Lancement
mongosh

→

~/.mongoshrc.js
exécuté s'il est présent

→

Prompt
affiché ensuite

Avec un fichier .js en ligne de commande, il est exécuté puis .mongoshrc.js ; `--norc` désactive ce chargement.

Personnaliser le prompt

```
host = db.serverStatus().host;
prompt = function() {
  return db.getName() + "@" + host + "$ ";
}
```

Nombre de documents affichés

```
DBQuery.shellBatchSize = 10;
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Le driver C#

MODULE 3

Gestion des drivers

Ce chapitre : **utiliser MongoDB depuis une application C#**.

1. **Composantes et compatibilité**
   Le driver .NET et les versions supportées.

2. **Connexion et choix d'une base**
   MongoClient, base et collection.

3. **CRUD avec le driver et LINQ**
   Créer, lire, modifier et supprimer en C#.

4. **Entity Framework**
   Intégration dans l'ORM, usage et limites.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Les drivers disponibles pour MongoDB.

Chapitre 1

Les drivers disponibles pour MongoDB

MongoDB Inc. maintient des **drivers officiels** pour 12 langages.

C	**C++**	**C#**	**Go**
Java	**Node.js**	**PHP**	**Python**
Ruby	**Rust**	**Scala**	**Swift**

C# est le driver utilisé dans ce cours. Des drivers **communautaires** existent aussi pour d'autres langages.

Le driver C# – MongoDB.Driver

Fonctionnalités clés du driver officiel pour .NET.

`MongoDB.Driver` est la bibliothèque officielle développée par MongoDB Inc. : une API typée pour .NET.

Connexion
simplifiée à la base

Accès direct
aux bases et collections

CRUD asynchrone
async / await

LINQ
support complet

Avancé
transactions, index, agrégations

Compatibilité des versions

Choisissez la version du driver selon **votre serveur MongoDB** et **votre version de .NET**.

Avec le serveur MongoDB

Serveur MongoDB	Driver C# minimum
5.0	`≥ 2.14`
6.0	`≥ 2.18`
6.1	`≥ 2.19`
7.0	`≥ 2.22`
8.0	`≥ 2.25`

Avec .NET

Driver C#	.NET supporté
`≤ 2.14`	.NET Framework 4.5 / .NET Core 3.1
`≥ 2.15`	.NET 5, 6, 7
`≥ 2.22`	.NET 8

En pratique : prenez la dernière version stable du driver.

Installer le driver MongoDB pour .NET

Le driver s'installe depuis **NuGet**, en ligne de commande ou depuis Visual Studio.

En ligne de commande CLI .NET ou console NuGet

```
dotnet add package MongoDB.Driver

# ou dans la console du Gestionnaire de packages
Install-Package MongoDB.Driver
```

Ou via le Gestionnaire NuGet



.NET

.NET 6+, .NET Core et .NET Framework 4.6+

Multiplateforme

Windows, Linux et macOS

Sync et async

API async / await pour de hautes performances

Connexion avec MongoClient

La classe `MongoClient` ouvre la connexion au serveur à partir d'une URI.

Créer le client serveur local

```
using MongoDB.Driver;

var client = new MongoClient("mongodb://localhost:27017");
```

Format de l'URI les crochets indiquent les parties optionnelles

```
mongodb://[utilisateur:motdepasse@]hôte[:port]/[base]?options
```

Bonne pratique : créez un seul MongoClient pour toute l'application (singleton) : il gère lui-même le pool de connexions.

Sélection d'une base et d'une collection

Depuis le client, on accède à une base puis à une collection : **elles sont créées à la première écriture**.

Accéder à une base GetDatabase()

```
var database = client.GetDatabase("MaBase");
```

Accéder à une collection GetCollection<T>() : BsonDocument ou une classe POCO

```
var collection = database.GetCollection<BsonDocument>("Utilisateurs");
var people     = database.GetCollection<Person>("people");
```

Créer une collection explicitement utile pour passer des options (validation, capped…)

```
await database.CreateCollectionAsync("Utilisateurs");
```

Premier programme C# avec MongoDB

Un **`Program.cs`** minimal : se connecter, insérer un document, puis le relire.

```
using MongoDB.Bson;
using MongoDB.Driver;

var client = new MongoClient("mongodb://localhost:27017");
var db     = client.GetDatabase("MaBase");
var users  = db.GetCollection<BsonDocument>("Utilisateurs");

// Insérer
var doc = new BsonDocument { { "name", "Alice" }, { "age", 30 } };
await users.InsertOneAsync(doc);

// Relire
var filter = Builders<BsonDocument>.Filter.Eq("name", "Alice");
var found  = await users.Find(filter).FirstOrDefaultAsync();
Console.WriteLine(found);
```

Résultat console

```
{ "_id" : ObjectId("…"),
  "name" : "Alice",
  "age" : 30 }
```

À retenir

La base et la collection sont créées à la première insertion.

Le driver ajoute _id s'il manque.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Les instructions CRUD en C# : Create, Read, Update, Delete

Chapitre 2

Les instructions CRUD en C# – Trois approches

- Chaque exemple est écrit de trois façons, avec cette classe POCO et ces collections :

Classe POCO mappée sur « people »

```
public class Person
{
    public ObjectId Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
    public string City { get; set; }
    public string Email { get; set; }
    public string Status { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

Mise en place namespaces et mapping

```
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Conventions;
using MongoDB.Driver;    // Builders<T>, AsQueryable()

// Name (C#) <-> name (MongoDB)
var pack = new ConventionPack
    { new CamelCaseElementNameConvention() };
ConventionRegistry.Register("camelCase", pack, t => true);
```

Collections

```
var collection = db.GetCollection<BsonDocument>("people");   // exemples BsonDocument
var people     = db.GetCollection<Person>("people");         // exemples Builders + POCO et LINQ
```

Requêtes LINQ

LINQ interroge les collections **directement en C#**, avec une syntaxe lisible et typée.

Requête LINQ C#

```
var parisiens = people.AsQueryable()
    .Where(p => p.City == "Paris")
    .OrderBy(p => p.Name)
    .Select(p => p.Name)
    .ToList();
```

Pipeline envoyé traduit par le driver

```
[
  { $match:   { City: "Paris" } },
  { $sort:    { Name: 1 } },
  { $project: { Name: 1 } }
]
```

✓ Typée

Une faute sur un champ est détectée à la compilation.

✓ Traduite

Le driver convertit l'expression en requête MongoDB.

✓ Familière

Where, Select, OrderBy, GroupBy, Count…

Les instructions CRUD en C# – Create

- InsertOne insère un document ; la variante InsertOneAsync s'utilise avec await.

BsonDocument non typé, proche de la syntaxe du shell

```
var doc = new BsonDocument { { "name", "John" }, { "age", 28 } };
collection.InsertOne(doc);
// ou en asynchrone : await collection.InsertOneAsync(doc);
```

POCO objet C# typé

```
var john = new Person { Name = "John", Age = 28 };
people.InsertOne(john);   // john.Id est renseigné après l'insertion
// ou en asynchrone : await people.InsertOneAsync(john);
```

LINQ non applicable à l'écriture

```
// AsQueryable() ne sert qu'à lire : il n'existe pas d'insertion LINQ.
// L'insertion typée passe par le POCO (exemple ci-dessus).
```

Les instructions CRUD en C# – Create

- InsertMany insère une liste de documents en une seule opération.

BsonDocument non typé, proche de la syntaxe du shell

```
var users = new List<BsonDocument>
{
    new BsonDocument { { "name", "John" }, { "age", 28 } },
    new BsonDocument { { "name", "Albert" }, { "age", 32 } }
};
await collection.InsertManyAsync(users);
```

POCO objet C# typé

```
var users = new List<Person>
{
    new Person { Name = "John", Age = 28 },
    new Person { Name = "Albert", Age = 32 }
};
await people.InsertManyAsync(users);
```

Si « _id » n'est pas précisé, le driver génère automatiquement un ObjectId. LINQ ne permet pas d'insérer.

Les instructions CRUD en C# – Read

- Find renvoie les documents du filtre ; un filtre vide renvoie toute la collection.

BsonDocument non typé, proche de la syntaxe du shell

```
var all = collection.Find(new BsonDocument()).ToList();
var filter = new BsonDocument
{
    { "age", new BsonDocument { { "$gte", 18 }, { "$lt", 65 } } },
    { "city", "Paris" }
};
var results = collection.Find(filter).ToList();
```

Builders + POCO typé, API fluide Builders<T>

```
var all = people.Find(Builders<Person>.Filter.Empty).ToList();
var f = Builders<Person>.Filter;
var filter = f.Gte(p => p.Age, 18) & f.Lt(p => p.Age, 65) & f.Eq(p => p.City, "Paris");
var results = people.Find(filter).ToList();
```

LINQ typé, expressions lambda C#

```
var all = people.AsQueryable().ToList();
var results = people.AsQueryable()
    .Where(p => p.Age >= 18 && p.Age < 65 && p.City == "Paris")
    .ToList();
```

Les instructions CRUD en C# – Read

- Une projection limite les champs renvoyés pour chaque document.

BsonDocument non typé, proche de la syntaxe du shell

```
var projection = new BsonDocument { { "name", 1 }, { "age", 1 }, { "_id", 0 } };
var results = collection.Find(new BsonDocument()).Project(projection).ToList();
```

Builders + POCO typé, API fluide Builders<T>

```
var projection = Builders<Person>.Projection
    .Include(p => p.Name).Include(p => p.Age).Exclude(p => p.Id);
var results = people.Find(Builders<Person>.Filter.Empty)
    .Project(projection).ToList();    // List<BsonDocument>
```

LINQ typé, expressions lambda C#

```
var results = people.AsQueryable()
    .Select(p => new { p.Name, p.Age })    // objet anonyme typé
    .ToList();
```

Les instructions CRUD en C# – Read

- Le tri est défini avant la lecture des résultats : 1 = croissant, -1 = décroissant.

BsonDocument non typé, proche de la syntaxe du shell

```
var sort = new BsonDocument { { "name", 1 }, { "age", -1 } };
var results = collection.Find(new BsonDocument()).Sort(sort).ToList();
```

Builders + POCO typé, API fluide Builders<T>

```
var sort = Builders<Person>.Sort.Ascending(p => p.Name).Descending(p => p.Age);
var results = people.Find(Builders<Person>.Filter.Empty).Sort(sort).ToList();
```

LINQ typé, expressions lambda C#

```
var results = people.AsQueryable()
    .OrderBy(p => p.Name).ThenByDescending(p => p.Age)
    .ToList();
```

Les instructions CRUD en C# – Read

- Skip et Limit (Take en LINQ) découpent les résultats en pages ; CountDocuments compte sans charger les documents.

BsonDocument non typé, proche de la syntaxe du shell

```
var page = collection.Find(new BsonDocument()).Skip(20).Limit(10).ToList();
long total = collection.CountDocuments(new BsonDocument { { "city", "Paris" } });
```

Builders + POCO typé, API fluide Builders<T>

```
var page = people.Find(Builders<Person>.Filter.Empty).Skip(20).Limit(10).ToList();
long total = people.CountDocuments(Builders<Person>.Filter.Eq(p => p.City, "Paris"));
```

LINQ typé, expressions lambda C#

```
var page = people.AsQueryable().Skip(20).Take(10).ToList();
int total = people.AsQueryable().Count(p => p.City == "Paris");
```

Les instructions CRUD en C# – Read

- FirstOrDefault renvoie le premier document trouvé, ou null s'il n'y en a aucun.

BsonDocument non typé, proche de la syntaxe du shell

```
var filter = new BsonDocument { { "name", "Alice" } };
BsonDocument alice = collection.Find(filter).FirstOrDefault();
```

Builders + POCO typé, API fluide Builders<T>

```
var filter = Builders<Person>.Filter.Eq(p => p.Name, "Alice");
Person alice = people.Find(filter).FirstOrDefault();
```

LINQ typé, expressions lambda C#

```
Person alice = people.AsQueryable().FirstOrDefault(p => p.Name == "Alice");
```

Les instructions CRUD en C# – Read

- BsonDocument se lit par l'indexeur ; un POCO, par des propriétés typées.

BsonDocument non typé, proche de la syntaxe du shell

```
foreach (var doc in collection.Find(new BsonDocument()).ToList())
{
    string name = doc["name"].AsString;          // conversion explicite : AsString,
    int age = doc["age"].AsInt32;                // AsInt32, AsDouble, AsBoolean,
    DateTime created = doc["createdAt"].ToUniversalTime();   // AsBsonDocument, AsBsonArray
}
```

Builders + POCO typé, API fluide Builders<T>

```
foreach (var p in people.Find(Builders<Person>.Filter.Empty).ToList())
{
    Console.WriteLine($"{p.Name} ({p.Age} ans)");   // propriétés typées, sans conversion
}
```

LINQ typé, expressions lambda C#

```
var names = people.AsQueryable().Select(p => p.Name).ToList();
names.ForEach(Console.WriteLine);
```

Les instructions CRUD en C# – Update

- ReplaceOne remplace un document entier ; le champ « _id » est conservé.

BsonDocument non typé, proche de la syntaxe du shell

```
var filter = new BsonDocument { { "name", "Alice" } };
var replacement = new BsonDocument { { "name", "Alice" }, { "age", 30 }, { "city", "Paris" } };
collection.ReplaceOne(filter, replacement);
```

Builders + POCO typé, API fluide Builders<T>

```
var filter = Builders<Person>.Filter.Eq(p => p.Name, "Alice");
var alice = people.Find(filter).First();   // le POCO lu conserve son Id
alice.Age = 30; alice.City = "Paris";
people.ReplaceOne(filter, alice);
```

LINQ typé, expressions lambda C#

```
var alice = people.AsQueryable().First(p => p.Name == "Alice");
alice.Age = 30; alice.City = "Paris";
people.ReplaceOne(p => p.Id == alice.Id, alice);   // filtre en expression lambda
```

Les instructions CRUD en C# – Update

- UpdateOne modifie certains champs ($set, $inc, $unset…) ; UpdateMany s'écrit de la même façon.

BsonDocument non typé, proche de la syntaxe du shell

```
var filter = new BsonDocument { { "name", "Alice" } };
var update = new BsonDocument { { "$set", new BsonDocument { { "age", 30 }, { "city", "Paris" } } } };
collection.UpdateOne(filter, update);
```

Builders + POCO typé, API fluide Builders<T>

```
var filter = Builders<Person>.Filter.Eq(p => p.Name, "Alice");
var update = Builders<Person>.Update.Set(p => p.Age, 30).Set(p => p.City, "Paris");
people.UpdateOne(filter, update);
```

LINQ typé, expressions lambda C#

```
// LINQ exprime le filtre ; la modification reste une UpdateDefinition
people.UpdateOne(p => p.Name == "Alice",
    Builders<Person>.Update.Set(p => p.Age, 30).Set(p => p.City, "Paris"));
```

Les instructions CRUD en C# – Update

- Avec IsUpsert = true, le document est créé s'il n'existe pas encore.

BsonDocument non typé, proche de la syntaxe du shell

```
var filter = new BsonDocument { { "email", "bob@acme.com" } };
var update = new BsonDocument
{
    { "$set", new BsonDocument { { "name", "Bob" } } },
    { "$setOnInsert", new BsonDocument { { "createdAt", DateTime.UtcNow } } }
};
collection.UpdateOne(filter, update, new UpdateOptions { IsUpsert = true });
```

Builders + POCO typé, API fluide Builders<T>

```
var filter = Builders<Person>.Filter.Eq(p => p.Email, "bob@acme.com");
var update = Builders<Person>.Update
    .Set(p => p.Name, "Bob")
    .SetOnInsert(p => p.CreatedAt, DateTime.UtcNow);
people.UpdateOne(filter, update, new UpdateOptions { IsUpsert = true });
```

LINQ typé, expressions lambda C#

```
people.UpdateOne(p => p.Email == "bob@acme.com", update,   // même update que ci-dessus
    new UpdateOptions { IsUpsert = true });
```

Les instructions CRUD en C# – Delete

- DeleteOne supprime le premier document trouvé, DeleteMany tous les documents du filtre.

BsonDocument non typé, proche de la syntaxe du shell

```
collection.DeleteOne(new BsonDocument { { "name", "Alice" } });
collection.DeleteMany(new BsonDocument { { "status", "inactive" } });
```

Builders + POCO typé, API fluide Builders<T>

```
people.DeleteOne(Builders<Person>.Filter.Eq(p => p.Name, "Alice"));
people.DeleteMany(Builders<Person>.Filter.Eq(p => p.Status, "inactive"));
```

LINQ typé, expressions lambda C#

```
people.DeleteOne(p => p.Name == "Alice");
people.DeleteMany(p => p.Status == "inactive");
```

Les instructions CRUD en C# – Delete

- Plusieurs conditions se combinent dans un même filtre (ici : mineurs habitant Paris).

BsonDocument non typé, proche de la syntaxe du shell

```
var filter = new BsonDocument { { "city", "Paris" }, { "age", new BsonDocument("$lt", 18) } };
collection.DeleteMany(filter);
```

Builders + POCO typé, API fluide Builders<T>

```
var f = Builders<Person>.Filter;
var filter = f.And(f.Eq(p => p.City, "Paris"), f.Lt(p => p.Age, 18));   // ou : f.Eq(...) & f.Lt(...)
people.DeleteMany(filter);
```

LINQ typé, expressions lambda C#

```
people.DeleteMany(p => p.City == "Paris" && p.Age < 18);
```

Les instructions CRUD en C# – Delete

- DeleteResult.DeletedCount indique le nombre de documents réellement supprimés.

BsonDocument non typé, proche de la syntaxe du shell

```
DeleteResult result = collection.DeleteMany(new BsonDocument { { "city", "Paris" } });
Console.WriteLine($"{result.DeletedCount} document(s) supprimé(s)");
```

Builders + POCO typé, API fluide Builders<T>

```
var result = await people.DeleteManyAsync(Builders<Person>.Filter.Eq(p => p.City, "Paris"));
Console.WriteLine($"{result.DeletedCount} document(s) supprimé(s)");
```

LINQ typé, expressions lambda C#

```
var result = await people.DeleteManyAsync(p => p.City == "Paris");
Console.WriteLine($"{result.DeletedCount} document(s) supprimé(s)");
```

Les instructions CRUD en C# – Delete

Bonnes pratiques : une suppression est **définitive**, il n'y a pas de retour arrière.

`DeleteOne`
Cibler précisément un document
ex. une transaction par son Id

`DeleteMany`
Nettoyer ou archiver en masse
ex. tous les comptes clôturés

Pas de rollback : sauvegardez ou archivez les documents avant de les supprimer.

Toujours tester le filtre avant même filtre pour compter, puis pour supprimer

```
var filter = Builders<Person>.Filter.Eq(p => p.Status, "inactive");

var nb = await people.CountDocumentsAsync(filter);       // 1. vérifier
Console.WriteLine($"{nb} documents vont être supprimés");

var result = await people.DeleteManyAsync(filter);       // 2. supprimer
Console.WriteLine($"{result.DeletedCount} supprimés");
```

Entity Framework & MongoDB

Une approche **ORM** (Object-Relational Mapping) accède aux données MongoDB via des **entités C# typées**.

Driver officiel `MongoDB.Driver`

La base de tout le cours : Builders, LINQ, BsonDocument.

ORM alternatifs compatibles NoSQL

`MongoFramework`

Style Entity Framework, au-dessus du driver

`MongoDB.EntityFrameworkCore`

Provider EF Core officiel (slide suivante)

`NoEntityFramework.MongoDB`

Bibliothèque communautaire

Entity Framework & MongoDB

Le provider **`MongoDB.EntityFrameworkCore`** permet d'utiliser EF Core avec MongoDB.

Exemple DbContext et requête LINQ

```
public class BanqueContext : DbContext
{
    public DbSet<User> Users { get; set; }

    protected override void OnConfiguring(
        DbContextOptionsBuilder options)
        => options.UseMongoDB(
            "mongodb://localhost:27017", "banque");
}

using var dbContext = new BanqueContext();
var users = dbContext.Users
    .Where(u => u.IsActive)
    .ToList();
```

Requêtes typées via LINQ

Le compilateur vérifie les noms de champs.

Sérialisation automatique

Les objets C# sont convertis en BSON, et inversement.

Pas de migration SQL

Le schéma suit la classe, sans script de migration.

Compatible .NET 8+

Le provider s'appuie sur EF Core 8.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

MODÉLISATION ET INDEXATION

MODULE 4

Modélisation et indexation

Ce module : **bien modéliser, puis rendre les requêtes rapides**.

1. **Schema design**
   Choisir la modélisation adaptée.
2. **Indexation**
   Créer les bons index.
3. **Optimisation et profiling**
   Repérer les requêtes lentes.
4. **Plans d'exécution**
   Lire le résultat de explain().
5. **Fonctionnement interne**
   Transactions, isolation, verrous.
6. **Performances**
   Mémoire, index en RAM, bonnes pratiques.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Le schéma design : réflexion sur la modélisation à utiliser

Chapitre 1

Le schéma de données

MongoDB **n'impose pas de schéma fixe** aux documents d'une collection.

Schéma flexible

Deux documents d'une même collection peuvent avoir des champs différents.

Structure similaire en pratique

Les documents d'une collection se ressemblent ; la flexibilité facilite les évolutions.

Le défi : trouver l'équilibre entre

- **Besoins de l'application**
- **Performances du moteur**
- **Accès aux données**

À la conception, partez de l'usage

Requêtes, mises à jour et traitements de l'application, ainsi que la structure propre aux données.

Les différents scénarios de schémas

La décision clé : **comment représenter les relations** entre les données.

1. Les documents intégrés

Les données liées sont dans le même document.

```
// clients
{
  _id: "C001",
  nom: "Anas Ben Yaiche",
  adresse: {
    ville: "Paris",
    cp: "75011"
  }
}
```

→ **une seule lecture**

2. Les références

Un document pointe vers un autre par son _id.

```
// clients
{ _id: "C001", nom: "Anas Ben Yaiche" }

// comptes
{
  _id: "FR-0042",
  clientId: "C001",   // référence
  solde: 1250
}
```

→ **pas de duplication**

Les documents intégrés

Les données liées sont stockées **dans un seul document**.

Imbriquer

Un sous-document dans un champ, ou une liste de sous-documents dans un tableau.

Dénormaliser

Les données associées sont dupliquées là où on les lit, au lieu d'être réparties.

Une seule opération

L'application lit et modifie les données liées en une seule requête.



Les documents intégrés

On intègre les données quand elles sont **toujours lues avec leur parent**.

Relation « A UN » One-to-One

Un client a une seule adresse.

```
{
  _id: "C001",
  nom: "Anas Ben Yaiche",
  adresse: {
    rue: "12 rue Oberkampf",
    ville: "Paris"
  }
}
```

Relation « A DES » One-to-Many

Les enfants dépendent du parent : une commande et ses lignes.

```
{
  _id: "CMD-881",
  client: "C001",
  lignes: [
    { produit: "Carte Gold", qte: 1 },
    { produit: "Chéquier",   qte: 2 }
  ]
}
```

À éviter si le tableau grossit sans limite : un document est limité à 16 Mo.

Les références

Une référence est un **lien d'un document vers un autre**, par son identifiant.

Un lien, pas une copie

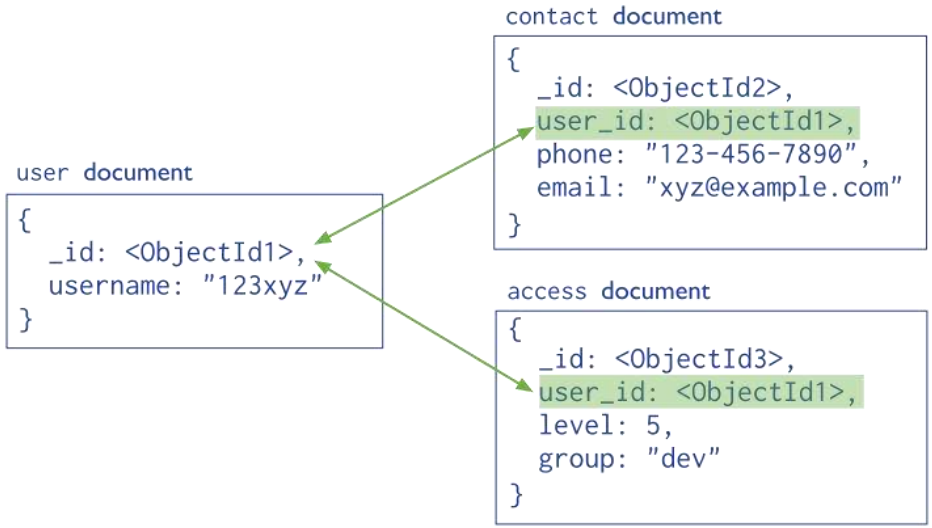
Le document contient le _id d'un autre document (ex : user_id).

Résolue par l'application

Deux requêtes, ou une jointure $lookup dans un pipeline.

Modèle normalisé

Chaque donnée n'existe qu'à un seul endroit.



contact et access pointent vers user par user_id

Les références

On utilise des références dans **trois situations**.

1. Trop de duplication

L'intégration copierait les mêmes données partout, pour un gain de lecture faible.

```
// le même client
// dans 10 000 comptes ✗
{ clientId: "C001" }  ✓
```

2. Relation « Many-to-Many »

Plusieurs entités de chaque côté.

```
// un compte, plusieurs
// titulaires, et inversement
{ titulaires:
    ["C001", "C002"] }
```

3. Grande hiérarchie

Des niveaux profonds ou nombreux.

```
// agence → région
{ _id: "AG-12",
  parent: "REG-IDF" }
```

La validation de document

Depuis la 3.2, MongoDB peut **valider les documents** à la création et à la mise à jour.

Par collection

À la création, ou sur une collection existante avec la commande collMod.

Règles

Tous les opérateurs de recherche, sauf $near, $nearSphere, $text et $where.

Paramètre	Valeur	Effet
`validationLevel`	`strict`	Validation sur toutes les créations et mises à jour
	`moderate`	Seulement sur les documents déjà valides avant la modification
`validationAction`	`error`	Erreur : le document fautif est rejeté
	`warn`	Avertissement dans les logs, l'écriture est acceptée

La validation de document

Un validateur avec des **opérateurs de requête** : le document doit respecter au moins une règle.

```
db.createCollection("contacts", {
  validator: {
    $or: [                                  // 1
      { phone:  { $type: "string" } },
      { email:  { $regex: /@mongodb\.com$/ } },
      { status: { $in: ["Unknown", "Incomplete"] } }
    ]
  },
  validationAction: "warn",                 // 2
  validationLevel:  "moderate"              // 3
})
```

1. **`$or`**
   Au moins une règle doit être vraie : un téléphone texte, un email @mongodb.com ou un statut connu.

2. **`validationAction: "warn"`**
   Un document invalide est quand même inséré, mais un avertissement est écrit dans le journal.

3. **`validationLevel: "moderate"`**
   Les documents existants déjà invalides ne sont pas contrôlés lors de leurs mises à jour.

La validation de document

`$jsonSchema` (≥ 3.6) exprime des règles par champ, regroupées selon le type de valeur.

Tous les types

`bsonType`	type imposé (alias ou tableau d'alias)
`enum`	liste des valeurs autorisées
`allOf`	respecter tous les schémas
`anyOf`	respecter au moins un schéma
`oneOf`	respecter exactement un schéma
`not`	ne pas respecter le schéma

Nombres

`multipleOf`	multiple de la valeur
`minimum`	valeur minimale
`maximum`	valeur maximale
`exclusiveMinimum`	minimum exclu (> au lieu de ≥)
`exclusiveMaximum`	maximum exclu (< au lieu de ≤)

La validation de document

Règles `$jsonSchema` propres aux chaînes, aux objets et aux tableaux.

Chaînes

`minLength`	nombre minimum de caractères
`maxLength`	nombre maximum de caractères
`pattern`	expression régulière à respecter

Tableaux

`items`	schéma (ou liste de schémas) des éléments
`minItems`	nombre minimum d'éléments
`maxItems`	nombre maximum d'éléments
`uniqueItems`	éléments tous différents

Objets

`properties`	schéma JSON de chaque champ
`required`	champs obligatoires
`additionalProperties`	accepte ou non d'autres champs
`minProperties`	nombre minimum de champs
`maxProperties`	nombre maximum de champs
`patternProperties`	comme properties, noms de champs en regex

La validation de document

Avec **`$jsonSchema`**, on décrit la structure attendue du document.

```
db.createCollection("diplomes", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["nom", "annee", "filiere"], // 1
    properties: {
      _id:     { bsonType: "objectId" },
      nom:     { bsonType: "string" },
      genre:   { bsonType: "string" },
      annee:   { bsonType: "int",            // 2
                 minimum: 1958, maximum: 2017 },
      filiere: { enum: ["Maths", "Lettres",  // 3
                  "Informatique", "Histoire", null] }
    },
    additionalProperties: false            // 4
  } }
})
```

1. **`required`**
   Ces champs doivent être présents.

2. **`bsonType + minimum / maximum`**
   annee est un entier entre 1958 et 2017.

3. **`enum`**
   filiere ne prend qu'une des valeurs listées.

4. **`additionalProperties: false`**
   Aucun champ non déclaré n'est accepté.

Avec additionalProperties: false, **déclarez _id**, sinon tout insert est refusé.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Validation avec C#

Chapitre 2

Validation des documents avec C#

La validation a lieu **avant l'insertion**, grâce aux attributs `DataAnnotations` sur la classe POCO.

Exemple classe Client

```
public class Client
{
    [Required]                  // 1
    public string Name { get; set; }

    [Range(18, 120)]            // 2
    public int Age { get; set; }

    [EmailAddress]              // 3
    public string Email { get; set; }

    [StringLength(50)]          // 4
    public string City { get; set; }
}
```

1. **`[Required]`**
   Champ obligatoire
   la valeur ne peut pas être null

2. **`[Range(min, max)]`**
   Plage de valeurs autorisée
   ici un âge entre 18 et 120

3. **`[EmailAddress]`**
   Format d'e-mail valide
   refuse « anas.mail.fr »

4. **`[StringLength(max)]`**
   Longueur maximale d'une chaîne
   ici 50 caractères au plus

Déclencher la validation le driver ne lit pas ces attributs tout seul

```
Validator.ValidateObject(client, new ValidationContext(client), true);  // sinon ValidationException
await clients.InsertOneAsync(client);
```

Validation des documents avec C# (Validator)

`Validator.TryValidateObject` contrôle l'objet **avant l'insertion**, sans lever d'exception.

Code un client volontairement invalide

```
var client  = new Client { Name = "", Email = "test@", Age = 10 };
var context = new ValidationContext(client);
var results = new List<ValidationResult>();

bool isValid = Validator.TryValidateObject(
    client, context, results, true);   // true = tous les attributs

if (!isValid)
{
    foreach (var error in results)
        Console.WriteLine($"Erreur : {error.ErrorMessage}");
}
else
    await clients.InsertOneAsync(client);
```

Sortie console

```
Erreur : Name
  est obligatoire
Erreur : Email
  n'est pas valide
Erreur : Age
  doit être entre 18 et 120
```

Messages issus des attributs de la classe Client.

`TryValidateObject` renvoie un booléen ;
`ValidateObject` lève une exception.

Validation manuelle et logique métier

Pour les règles impossibles à exprimer avec `[Required]` ou `[Range]`, on code la validation.

Unicité
Vérifier qu'un e-mail n'existe pas déjà.

Cohérence
Contrôler deux champs : StartDate < EndDate.

Format
Numéro de série, code client…

Exemple dates incohérentes et e-mail invalide

```
var user = new User { Email = "test", StartDate = DateTime.Today,
                      EndDate = DateTime.Today.AddDays(-1) };
var errors = user.Validate();   // liste des règles non respectées
```

La validation C# s'ajoute à celle de MongoDB ($jsonSchema) : **elle ne la remplace pas**.

Validation côté MongoDB (Schema Validator)

MongoDB contrôle lui-même chaque écriture grâce à **quatre paramètres** de la collection.

Exemple collection clients

```
db.createCollection("clients", {
  validator: {                    // 1
    $jsonSchema: {                // 2
      bsonType: "object",
      required: ["Name", "Email"],
      properties: {
        Email: { bsonType: "string",
                 pattern: "@" }
      }
    }
  },
  validationAction: "error",      // 3
  validationLevel: "strict"       // 4
})
```

1 `validator`
Définit les règles
$jsonSchema et opérateurs de requête

2 `$jsonSchema`
Décrit la structure du document
types, champs requis, formats

3 `validationAction`
Que faire si la règle échoue ?
`"error"` refuse · `"warn"` journalise

4 `validationLevel`
Quels documents contrôler ?
`"strict"` tous · `"moderate"` déjà valides

Validateur via C#

En C#, on décrit le schéma en `BsonDocument`, puis on l'applique à la création de la collection.

1. Décrire le schéma $jsonSchema

```
var schema = new BsonDocument
{
  { "bsonType", "object" },
  { "required", new BsonArray { "name", "email", "age", "createdAt" } },
  { "properties", new BsonDocument {
      { "name",  new BsonDocument { { "bsonType", "string" }, { "minLength", 2 } } },
      { "email", new BsonDocument { { "bsonType", "string" }, { "pattern", "^[^@\\s]+@[^@\\s]+\\.[^@\\s]+$" } } },
      { "age",   new BsonDocument { { "bsonType", "int" }, { "minimum", 18 }, { "maximum", 120 } } },
      { "createdAt", new BsonDocument { { "bsonType", "date" } } }
  }}
};
```

2. Créer la collection avec le validateur

```
await database.CreateCollectionAsync("users", new CreateCollectionOptions<BsonDocument>
{
    Validator = new BsonDocument("$jsonSchema", schema),
    ValidationLevel = DocumentValidationLevel.Strict,
    ValidationAction = DocumentValidationAction.Error
});
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

- L’indexation sous MongoDB

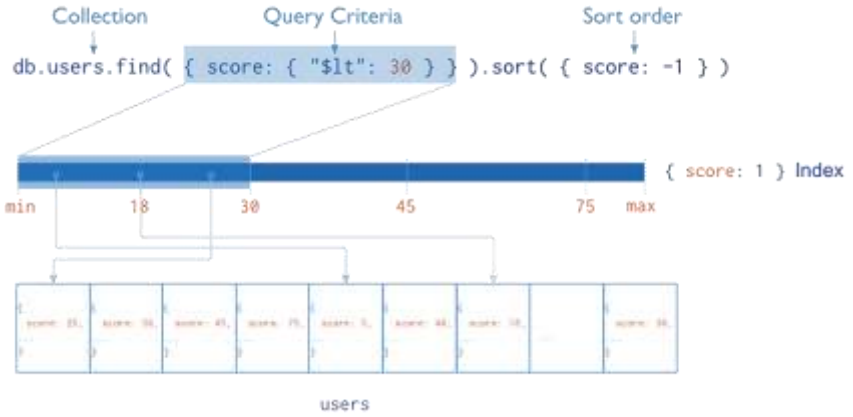
Chapitre 3

Les index

Les index permettent une **exécution efficace des requêtes** dans MongoDB.

Sans index
MongoDB parcourt tous les documents de la collection.

Avec un index adapté
Le nombre de documents à inspecter diminue.



Un index stocke les valeurs d'un ou plusieurs champs **triées dans l'ordre**.

Requêtes optimisées

✓ Égalité de valeurs

✓ Plage de valeurs

✓ Tris

Index par défaut et création d'index

Chaque collection a déjà un index **unique sur _id** : chaque identifiant y est unique.

Créer un index via le shell

```
db.<collection>.createIndex(<clés>, <options>)
```

`<clés>`	un document : le ou les champs de l'index et le type de chaque champ
`<options>`	un document optionnel : nom, unique, partiel, TTL…

Exemples

```
db.produits.createIndex({ prix: 1 })        // croissant sur prix
db.produits.createIndex({ _id: "hashed" })   // hashé sur _id
```

Types d’index

Trois types d'index pour **un champ, plusieurs champs ou un tableau**.

Un seul champ

le type d'index le plus simple

```
db.produits.createIndex(
  { prix: 1 })
```

Le sens a peu d'importance :
MongoDB parcourt l'index dans les deux directions pour un tri.

Composé

s'appuie sur plusieurs champs

```
db.users.createIndex(
  { userid: 1, score: -1 })
```

Le sens compte.

✓ `{ userid: 1, score: -1 }`
✓ `{ userid: -1, score: 1 }`
✗ `{ userid: 1, score: 1 }`
✗ `{ userid: -1, score: -1 }`

Multiclés

le contenu d'un tableau

```
db.users.createIndex(
  { "addr.zip": 1 })

// document indexé
{ userid: 1,
  addr: [ { zip: "75017" },
          { zip: "75020" } ] }
```

Une entrée d'index par valeur du tableau.

Types d'index

Trois index spécialisés, chacun lié à **un type de recherche**.

Géospatial

coordonnées GPS

```
db.places.createIndex(
  { loc: "2dsphere" })
```

Permet les recherches géographiques : **`$near`**, **`$geoWithin`**…

Texte

un ou plusieurs champs texte

```
db.reviews.createIndex(
  { title: "text",
    description: "text" },
  { weights: { title: 5 },
    default_language: "french"
})
```

Recherche avec **`$text`**.
Un seul index texte par collection : il couvre tous ses champs.

Hashé

stocke le hash des valeurs

```
db.users.createIndex(
  { _id: "hashed" })
```

Sert de **clé de sharding hashée** : répartition équitable entre les shards.

Propriétés d'index

Des options de `createIndex()` changent ce que l'index accepte ou contient.

Unique

Rejette les valeurs en double, sur un champ ou une combinaison de champs.

Un seul document autorisé sans le champ (null).

```
db.members.createIndex({ "user_id": 1 }, { unique: true })
```

Clairsemé sparse

N'indexe que les documents où le champ est renseigné.

Avec unique : plusieurs documents sans le champ sont permis.

```
db.addresses.createIndex({ "xmpp_id": 1 },
  { sparse: true })
```

Partiel ≥ 3.2

N'indexe que les documents qui vérifient un filtre ($eq, $exists, $gt, $gte, $lt, $lte, $type, $and).

Économise stockage et temps de calcul.

```
db.restaurants.createIndex(
  { rating: 1, cuisine: 1, name: 1 },
  { partialFilterExpression: { rating: { $gt: 5 } } })
```

Propriétés d'index

Deux autres options : l'expiration automatique et le nom de l'index.

TTL time to live

Supprime les documents après un délai, compté depuis une date du document. Utile pour des logs ou des sessions.

Champ tableau : la date la plus ancienne compte.
Champ vide ou non-date : pas d'expiration.

```
db.eventlog.createIndex({ "lastModifiedDate": 1 },
  { expireAfterSeconds: 3600 })
```

Nom personnalisé

Donne un nom lisible à l'index.

Par défaut, MongoDB concatène les noms et sens des champs (ex. lastModifiedDate_1).

```
db.eventlog.createIndex({ ... }, { name: "le_nom" })
```

Wildcard index

Un index « joker » (**$****) couvre des champs **non connus à l'avance**.

Documents structure de userMetadata variable

```
{ "userMetadata": { "likes": ["dogs", "cats"] } }
{ "userMetadata": { "dislikes": "pickles" } }
{ "userMetadata": { "age": 45 } }
{ "userMetadata": "inactive" }
```

Syntaxe tous les sous-champs de userMetadata

```
db.userData.createIndex(
  { "userMetadata.$**": 1 }
)
```

Exclure des champs index sur tout le document, sauf fieldA et fieldB.fieldC

```
db.collection.createIndex(
  { "$**": 1 },
  { wildcardProjection: { "fieldA": 0, "fieldB.fieldC": 0 } }
)
```

Wildcard index

Un index « joker » a **plusieurs limites** :

- ✗ Une seule condition de filtre prise en charge
- ✗ Pas combinable avec un autre index joker dans une requête
- ✗ Clairsemé : inutile pour chercher l'absence d'un champ
- ✗ Ne peut pas servir de clé de shard
- ✗ Ne peut pas être composé
- ✗ Pas de propriété TTL ni Unique
- ✗ Pas de type 2d, 2dsphere ou hashed

Cas à part : l'index joker « text »

Il indexe toutes les données textuelles trouvées dans un document.

```
db.collection.createIndex({ "$**": "text" })
```

Index « timeseries » (>= 5.0)

Les collections timeseries acceptent des index secondaires, avec des limites **en partie levées en 6.0**.

5.0 – 5.x : les limites

- **Seuls timeField et metaField** (et les sous-champs du metaField) peuvent être indexés
- metaField : pas d'index **2d, 2dsphere, text** en 5.0 ; seul **text** reste interdit en 5.1
- Propriétés **TTL, unique et partial** non supportées

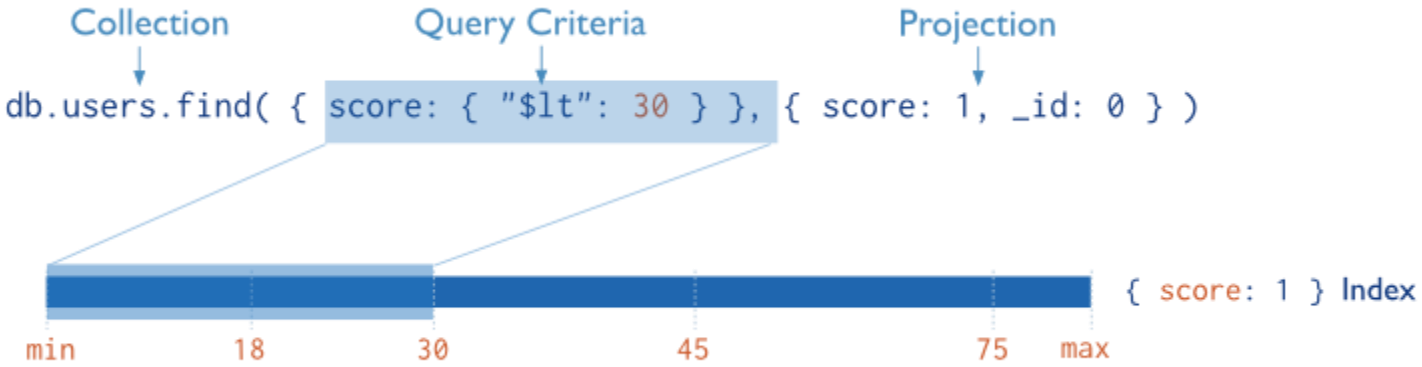
6.0 : ce qui devient possible

- Index **composé** sur timeField, metaField et les champs de mesure
- Index **partial et 2dsphere** sur n'importe quel champ
- **`$or`, `$in`, `$geoWithin`** utilisables sur un index partial

Requêtes couvertes

Une requête est **couverte** quand son filtre et sa projection n'utilisent que des champs d'un index.

MongoDB répond **directement depuis l'index**, sans lire aucun document ➔ grande efficacité.

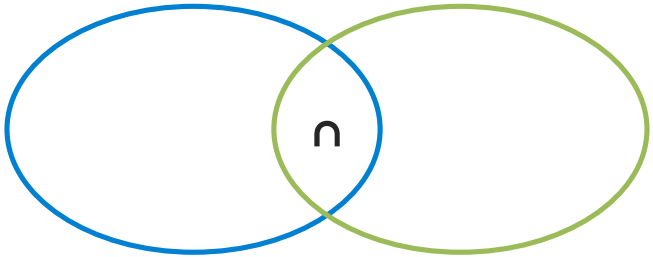


Dans l'exemple, `_id: 0` est obligatoire : l'index ne contient pas `_id`, sinon MongoDB doit lire le document.

Intersection d'index

MongoDB peut **combiner deux index** pour répondre à une même requête (collection orders).

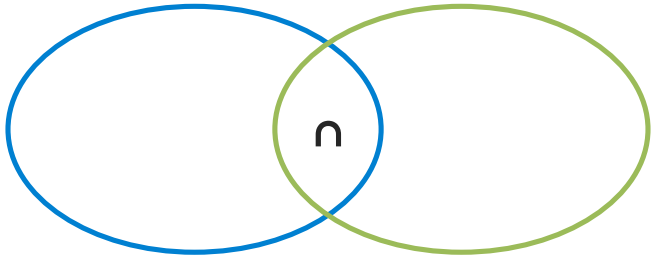
Deux index simples



`{ qty: 1 }` `{ item: 1 }`

```
db.orders.find({ item: "abc123", qty: { $gt: 15 } })
```

Un index composé utilisé par son début



`{ qty: 1 }` `{ status: 1, ord_date: -1 }`

```
db.orders.find({ qty: { $gt: 10 }, status: "A" })
```

Index composé : seul son préfixe status est utilisé

Un index composé reste souvent plus performant qu'une intersection : vérifiez avec `explain()`.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

L'indexation avec C#

Chapitre 4

Index avec C#

Sans index, MongoDB **lit chaque document** ; avec un index, il **va directement au bon**.

Sans index COLLSCAN

FR-0230	FR-0017	FR-0108
FR-0042	FR-0001	FR-0311

Les 6 documents sont lus pour en trouver 1.

Avec un index IXSCAN

Index trié

- FR-0001
- FR-0017
- **FR-0042**
- FR-0108

document

FR-0042

1 seul document lu.

Exemple retrouver un compte par son numéro

```
var compte = await accounts
    .Find(a => a.AccountNumber == "FR-0042").FirstOrDefaultAsync();
```

Sur 1 million de comptes : **1 000 000 documents lus sans index, 1 avec** (index sur AccountNumber).

Index avec C#

Un index permet à MongoDB de **localiser les documents sans parcourir toute la collection**.

Créer un index simple un seul champ, ordre croissant

```
var accounts = db.GetCollection<BankAccount>("accounts");

var keys = Builders<BankAccount>.IndexKeys.Ascending(a => a.AccountNumber);
await accounts.Indexes.CreateOneAsync(new CreateIndexModel<BankAccount>(keys));
```

Nommer l'index utile pour le retrouver ou le supprimer

```
var keys    = Builders<BankAccount>.IndexKeys.Ascending(a => a.ClientId);
var options = new CreateIndexOptions { Name = "idx_client_id" };
await accounts.Indexes.CreateOneAsync(new CreateIndexModel<BankAccount>(keys, options));
```

`Ascending` ou `Descending` : sur un index simple, l'ordre est indifférent pour les recherches.

Index avec C#

Un index peut porter sur **plusieurs champs** (composé) ou sur **un tableau** (multiclés).

Index composé `{ ClientId: 1, Date: -1 }`

ClientId	Date ↓
C001	2025-03-12
C001	2025-02-01
C001	2025-01-15
C002	2025-03-02
C002	2025-01-20

Le 1er champ regroupe, le 2e trie.

Index multiclés `{ Beneficiaries: 1 }`

```
{ Account: "FR-0042",
  Beneficiaries:
    ["Léa", "Marc"] }
```

→ `"Léa"` → FR-0042

→ `"Marc"` → FR-0042

Une entrée d'index par élément du tableau.

Exemples requêtes servies par ces index

```
// les 10 dernières transactions de C001 : filtre + tri servis par l'index composé
var dernieres = await transactions.Find(t => t.ClientId == "C001")
    .SortByDescending(t => t.Date).Limit(10).ToListAsync();
```

```
// les comptes dont Léa est bénéficiaire : servie par l'index multiclés
var comptes = await accounts
    .Find(Builders<BankAccount>.Filter.AnyEq(a => a.Beneficiaries, "Léa")).ToListAsync();
```

Index avec C#

Un index peut porter sur **plusieurs champs** ou sur **un tableau**.

Index composé plusieurs champs

Par client, puis du plus récent au plus ancien.

```
var transactions =
    db.GetCollection<Transaction>("transactions");

var keys = Builders<Transaction>.IndexKeys
    .Ascending(t => t.ClientId)
    .Descending(t => t.Date);

await transactions.Indexes.CreateOneAsync(
    new CreateIndexModel<Transaction>(keys));
```

Index multiclés champ de type tableau

Créé **automatiquement** : chaque élément de `Beneficiaries` est indexé.

```
// Beneficiaries est une List<string>
var keys = Builders<BankAccount>.IndexKeys
    .Ascending(a => a.Beneficiaries);

await accounts.Indexes.CreateOneAsync(
    new CreateIndexModel<BankAccount>(keys));
```

Ordre des champs : l'index composé sert aussi les requêtes sur son début (ici `ClientId` seul), pas sur `Date` seule.

Index avec C#

Les options changent **le comportement** de l'index, pas seulement sa vitesse.

Unique

Refuse un 2e document avec la même valeur.

✓ `insert FR-0042`

✗ `insert FR-0042`

`E11000 duplicate key`

```
try {
  await accounts
    .InsertOneAsync(c);
} catch (MongoWriteException)
{ /* numéro déjà pris */ }
```

Partiel

N'indexe que les documents qui respectent un filtre.

`FR-0001 · actif` indexé

`FR-0017 · inactif`

`FR-0042 · actif` indexé

`FR-0108 · inactif`

```
// filtre avec IsActive
// pour utiliser l'index
accounts.Find(a =>
  a.IsActive &&
  a.Balance > 1000)
```

TTL

Supprime les documents une fois leur date passée.



Vérification toutes les 60 s : la suppression n'est pas instantanée.

```
// valable 30 min
var s = new Session {
  ExpirationDate = DateTime
    .UtcNow.AddMinutes(30)
};
```

Index avec C#

Les options se passent dans **`CreateIndexOptions`**, en second paramètre du modèle.

Index unique refuse les doublons sur un champ critique

```
var keys    = Builders<BankAccount>.IndexKeys.Ascending(a => a.AccountNumber);
var options = new CreateIndexOptions { Name = "idx_unique_account_number", Unique = true };
await accounts.Indexes.CreateOneAsync(new CreateIndexModel<BankAccount>(keys, options));
```

Index partiel une partie des documents

```
var options = new CreateIndexOptions<BankAccount>
{
    Name = "idx_active_accounts",
    PartialFilterExpression =
        Builders<BankAccount>.Filter
            .Eq(a => a.IsActive, true)
};
```

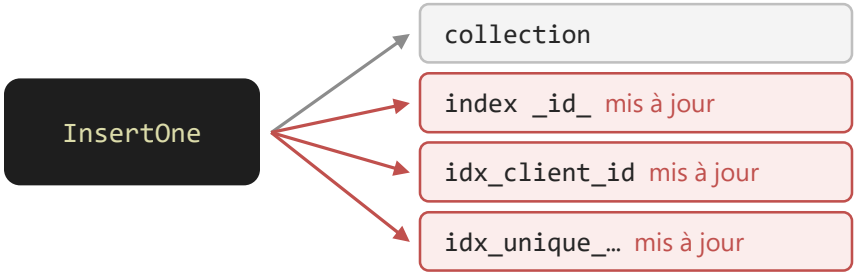
Index TTL suppression automatique

```
var keys = Builders<Session>.IndexKeys
    .Ascending(s => s.ExpirationDate);
var options = new CreateIndexOptions
{
    Name = "idx_ttl_session_expiration",
    ExpireAfter = TimeSpan.Zero
};  // supprimé dès ExpirationDate
```

Index avec C#

Un index accélère les lectures mais **coûte à chaque écriture** : on garde seulement ceux qui servent.

Le coût d'un index



1 écriture = 1 document + **chaque index** à mettre à jour.

Exemple : ce que renvoie ListAsync()

```
{ key: { _id: 1 }, name: "_id_" }
{ key: { ClientId: 1 }, name: "idx_client_id" }
{ key: { AccountNumber: 1 }, unique: true,
  name: "idx_unique_account_number" }
```

Le nom choisi sert ensuite à supprimer l'index.

La démarche

1. Lister
`ListAsync()`

2. Repérer l'inutile
jamais utilisé, doublon

3. Supprimer
`DropOneAsync("nom")`

Index avec C#

Vérifier les index existants et **supprimer ceux qui ne servent plus** : chaque index ralentit les écritures.

Lister les index d'une collection

```
var indexes = await accounts.Indexes.ListAsync();
await indexes.ForEachAsync(index => Console.WriteLine(index));
```

Supprimer un index par son nom

```
await accounts.Indexes
    .DropOneAsync("idx_unique_account_number");
```

Supprimer tous les index sauf _id

```
await accounts.Indexes.DropAllAsync();
```

Bon réflexe : nommez vos index à la création, pour les supprimer sans deviner leur nom généré (`AccountNumber_1`).

Index avec C#

Tous les index de la collection **en un seul appel**, au démarrage de l'application.

```
public static async Task CreerIndexAsync(IMongoCollection<BankAccount> accounts)
{
    var ix = Builders<BankAccount>.IndexKeys;
    var models = new[]
    {
        new CreateIndexModel<BankAccount>(ix.Ascending(a => a.AccountNumber),
            new CreateIndexOptions { Name = "idx_unique_account_number", Unique = true }),
        new CreateIndexModel<BankAccount>(ix.Ascending(a => a.ClientId),
            new CreateIndexOptions { Name = "idx_client_id" }),
        new CreateIndexModel<BankAccount>(ix.Descending(a => a.Balance),
            new CreateIndexOptions<BankAccount> { Name = "idx_active_balance",
                PartialFilterExpression = Builders<BankAccount>.Filter.Eq(a => a.IsActive, true) })
    };
    await accounts.Indexes.CreateManyAsync(models);
}
```

Sans risque : recréer un index identique (mêmes clés, mêmes options) ne fait rien.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

L'optimisation des requêtes et le profiling

Chapitre 5

Stratégies d’indexation

Les bons index dépendent de **l'usage réel de l'application**.

Trois facteurs à prendre en compte

Les requêtes attendues
filtres, projections et tris utilisés

Lectures / écritures
chaque index ralentit les écritures

La mémoire disponible
les index doivent tenir en RAM

Un index a un coût (création, maintenance) : il se justifie pour des requêtes **fréquentes** sur de **grands volumes**.

Bonne pratique
Tester plusieurs stratégies d'index sur une copie des données de production, puis garder la meilleure.

Créer des index pour supporter ses requêtes

Requête
filtre sur x et y

→

Index qui contient x et y
tous les champs filtrés

→

Pas de scan de la collection
➔ requête rapide

Quel index créer ?

- Toujours le même champ ➔ **index simple** `{ x: 1 }`
- Un champ, parfois un second ➔ **index composé** `{ x: 1, y: 1 }`, dans le même ordre

L'index { x, y, z } sert aussi ses préfixes

x: 1	y: 1	z: 1
✓ { x: 1, y: 1 }		
✓ { x: 1 }		

Filtre puis tri : l'ordre des champs de l'index compte

```
// index { x: 1, z: 1 }
db.collection.find({ x: 5 }).sort({ z: 1 })    // ✓ filtre sur x, puis tri sur z
db.collection.find({ z: 10 }).sort({ x: 1 })   // X non supporté par cet index
```

Index et tri des requêtes

Un index est utilisable
Le tri suit l'ordre de l'index : pas de tri à faire.

Aucun index utilisable
Tri en mémoire, abandonné au-delà de 100 Mo sauf `allowDiskUse()`

Index	Règle	✓ Tris supportés	✗ Tris non supportés
Champ unique `{ a: 1 }`	Les deux directions	`{ a: 1 }` `{ a: -1 }`	—
Composé `{ a: 1, b: 1 }`	Même ordre des champs	`{ a: 1, b: 1 }` `{ a: -1, b: -1 }`	`{ b: 1, a: 1 }`
Composé, sens mixtes `{ a: 1, b: -1 }`	Sens inversé sur tous les champs à la fois	`{ a: 1, b: -1 }` `{ a: -1, b: 1 }`	`{ a: -1, b: -1 }`
Composé, usage partiel `{ a: 1, b: 1, c: 1 }`	Préfixe de l'index	`{ a: 1 }` `{ a: 1, b: 1 }`	`{ b: 1, c: 1 }`

S’assurer que les index « tiennent » en RAM

Pour de bonnes performances, les index doivent **tenir entièrement en RAM** : pas d'accès disque.

Mesurer la taille des index d'une collection, en octets

```
db.<collection>.totalIndexSize()
```

Ce que la RAM doit contenir



RAM du serveur

Plusieurs collections

L'ensemble de leurs index doit tenir en RAM, pas seulement ceux d'une collection.

Jeu de travail

Gardez aussi de la place pour les documents que les requêtes manipulent.

Optimisation des requêtes

Les index sont la base ; **quatre leviers** réduisent ensuite le travail du serveur et du réseau.

1 Requête sélective

Un filtre précis limite les documents analysés et retournés.

2 Limiter les résultats

Moins de trafic réseau.

```
db.posts.find()
  .sort({ timestamp: -1 }).limit(10)
```

3 Projection

Récupérer uniquement les champs nécessaires.

```
db.users.find({}, { nom: 1, age: 1 })
```

4 hint()

Suggérer l'index à utiliser.

```
db.users.find({ nom: "DUPONT", age: { $gt: 50 } })
  .hint({ age: 1 })
```

Profiler de base de données

Le profiler collecte le détail des **écritures, curseurs et commandes**. Il s'active par base ou par instance, via le shell ou un driver.

Niveaux de profilage

Niveau	Données collectées
0	Aucune : profiler désactivé. Les requêtes lentes vont dans le log.
1	Requêtes lentes uniquement (au-delà du seuil).
2	Toutes les opérations.

Activer sur une base

```
db.setProfilingLevel(<niveau>, <seuil en ms>, <sampleRate>)
```

`sampleRate` : part des opérations profilées, de 0 à 1 (défaut 1, soit 100 %).

Profiler de base de données

Consulter, configurer et exploiter le profiler.

Connaître le niveau

Sur la base en cours :

```
db.getProfilingStatus()
```

Activer sur une instance

Dans le fichier de configuration :

```
operationProfiling:
  mode: slowOp        # niveau
  slowOpThresholdMs: 100   # seuil
```

Lire les résultats

Collection system.profile de la base, capped à 1 Mo par défaut.

Annoter une requête

Le commentaire apparaît dans system.profile :

```
db.produits.find().comment("liste produits")
```

Sharded cluster : activez le profiler sur chaque instance, mongos n'est qu'un routeur.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

L’analyse des plans d’exécution

Chapitre 6

Explication des requêtes

`explain()` montre comment MongoDB exécute une requête. Le détail dépend du niveau de verbosité.

Les deux sections retournées

`queryPlanner`
le plan choisi par l'optimiseur et la liste des plans rejetés

`executionStats`
le détail de l'exécution du plan vainqueur

	`queryPlanner`	`executionStats`	`allPlansExecution`
Choisit le plan vainqueur	✓	✓	✓
Exécute le plan	–	✓	✓
Retourne queryPlanner	✓	✓	✓
Retourne executionStats	–	✓	✓
Stats des plans rejetés	–	–	✓

Syntaxe `db.<collection>.<requête>.explain([verbosité])`

```
db.produits.find({ prix: { $gt: 100 } }).explain("executionStats")
```

Explication des requêtes

- Champs principaux du résultat (préfixe commun : **`explain.`**) :

Section « queryPlanner »	
`winningPlan`	Le plan d'exécution choisi par l'optimiseur
`winningPlan.stage`	Type d'étape : COLLSCAN (parcours collection), IXSCAN (index), FETCH (récupération), SHARD_MERGE…
`winningPlan.inputStage(s)`	Les étapes filles, exécutées juste avant
`rejectedPlans`	Les plans rejetés
Section « executionStats »	
`nReturned`	Nombre de documents retournés
`executionTimeMillis`	Temps d'exécution en millisecondes
`totalKeysExamined`	Nombre de clés d'index examinées
`totalDocsExamined`	Nombre de documents examinés
`executionStages(.inputStage…)`	Arbre des étapes exécutées — chaque étape porte ses propres totalKeysExamined / totalDocsExamined



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Fonctionnement de MongoDB : transaction, verrou

Chapitre 7

Transactions multi-documents

Depuis la 4.0, une transaction regroupe des écritures sur **plusieurs documents**, liées à une session.

`startSession()`
ouvrir une session

↓

`startTransaction()`
fixer readConcern et writeConcern

↓

Opérations
via les collections de la session

↓

`commit`
tout est validé

`abort`
tout est annulé

Replica set ≥ 4.0 · sharded cluster ≥ 4.2

```
const session = db.getMongo().startSession();
const employees = session.getDatabase("hr").employees;
const events = session.getDatabase("hr").events;
session.startTransaction({
  readConcern: { level: "snapshot" },
  writeConcern: { w: "majority" } });
try {
  employees.updateOne({ employee: 3 },
    { $set: { status: "Inactive" } });
  events.insertOne({ employee: 3,
    status: { new: "Inactive", old: "Active" } });
  session.commitTransaction();
} catch (error) {
  session.abortTransaction();
  throw error;
} finally {
  session.endSession();
}
```

Verrou

Une écriture sur un document est **atomique** : tout ou rien, jamais de vue incohérente pour les clients.

Niveaux de verrou ex. écriture sur une collection

- **Global** — IX
  - **Base de données** — IX
    - **Collection** — X
      - **Document** — *géré par WiredTiger*

Quatre modes

S **Partagé** lecture	**X** **Exclusif** écriture
IS **Intention S** un S est posé plus bas	**IX** **Intention X** un X est posé plus bas

- **Un S ou X à un niveau entraîne un IS ou IX sur tous les niveaux au-dessus.**
- Verrous distincts en lecture et en écriture : une lecture reste possible pendant un verrou d'écriture sur la ressource.

Verrou

Plusieurs verrous peuvent coexister sur une même ressource **si leurs modes sont compatibles**.

Compatibilité des modes

	IS	IX	S	X
IS	✓	✓	✓	✗
IX	✓	✓	✗	✗
S	✓	✗	✓	✗
X	✗	✗	✗	✗

File d'attente : un exemple



1. Accordés ensemble : tous les modes compatibles



2. Une fois relâchés : les X, l'un après l'autre



- Les demandes sont traitées dans leur ordre d'arrivée ; dès qu'une demande est accordée, **toutes les demandes compatibles le sont aussi**.

Verrou

WiredTiger gère la concurrence de façon **optimiste** pour la plupart des lectures et écritures.

Verrous d'intention
au niveau global, base et collection

→

Conflit entre 2 opérations
détecté sur un document

→

Une opération est retentée
plus tard, de façon transparente

Cas particuliers

- **Verrou global** : certaines opérations sur plusieurs bases
- **Verrou exclusif sur la base** : commandes d'administration comme `collMod`

Sharded cluster

- **Chaque shard** gère ses propres verrous.

Replica set

- **Le primaire** gère les verrous.
- **Les secondaires** rejouent le journal d'opérations dans l'ordre.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Transaction en C#

Chapitre 8

Transaction avec C#

Une transaction regroupe plusieurs opérations : **soit toutes réussissent, soit aucune n'est appliquée**.

1. Ouvrir une session

IClientSessionHandle : regroupe les opérations d'une même transaction

```
using var session = await client.StartSessionAsync();
```

2. Démarrer la transaction

les opérations suivantes sont exécutées de façon atomique

```
session.StartTransaction();
```

Prérequis : un replica set (≥ 4.0) ou un sharded cluster (≥ 4.2). Un serveur standalone ne gère pas les transactions.

Transaction avec C#

Chaque opération reçoit **la session en premier paramètre**, puis la transaction est validée.

3. Exécuter les opérations InsertOne, UpdateOne, DeleteOne... avec la session

```
await collection.InsertOneAsync(session, document);
await collection.UpdateOneAsync(session, filter, update);
await collection.DeleteOneAsync(session, filter);
```

4. Valider : CommitTransactionAsync() les modifications sont définitivement écrites dans la base

```
await session.CommitTransactionAsync();
```

Attention : une opération appelée sans la session s'exécute hors transaction et ne sera pas annulée en cas d'erreur.

Transaction avec C#

En cas d'erreur, `AbortTransactionAsync()` annule toutes les opérations de la transaction.

5. Annuler

```
await session
    .AbortTransactionAsync();
```

Rollback : aucune modification n'est appliquée à la base.

Si la transaction n'est ni validée ni annulée, elle est abandonnée à la fin de la session.

Exemple complet virement bancaire

```
using var session = await client.StartSessionAsync();
session.StartTransaction();
try
{
    // virement de 100 € de compteA vers compteB
    await comptes.UpdateOneAsync(session, c => c.Id == "compteA",
        Builders<Compte>.Update.Inc(c => c.Solde, -100));
    await comptes.UpdateOneAsync(session, c => c.Id == "compteB",
        Builders<Compte>.Update.Inc(c => c.Solde, 100));
    await session.CommitTransactionAsync();
}
catch
{
    await session.AbortTransactionAsync();   // rollback
    throw;
}
```

Transaction avec C#





CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

INTRODUCTION À LA RÉPLICATION ET AU SHARDING

MODULE 5

Introduction à la réplication et au Sharding

Deux mécanismes complémentaires : **copier les données** et **les répartir**.

Réplication
des copies pour la disponibilité

1. Introduction
2. La réplication Primary-Secondary
3. La bascule automatique du Primary
4. Les membres secondaires particuliers
5. Mise en place de la réplication

Sharding
une répartition pour la montée en charge

6. Introduction
7. Le sharding avec MongoDB
8. Mise en place du sharding
9. Répartition manuelle : zones



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Réplication : introduction

Chapitre 1

Réplication

La réplication **synchronise les données sur plusieurs serveurs** (disponible depuis la version 1.6).

Redondance

Les données existent en plusieurs copies.

Haute disponibilité

Le service continue si un serveur tombe.

Lectures réparties

Possible sur plusieurs serveurs, mais pas toujours une bonne idée.

Copies pour des besoins spécifiques

`hidden`

Membre invisible pour les applications : idéal pour le reporting.

`delayed`

Copie en retard volontaire : permet un retour arrière après une mise en production.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

La réplication Primary-Secondary

Chapitre 2

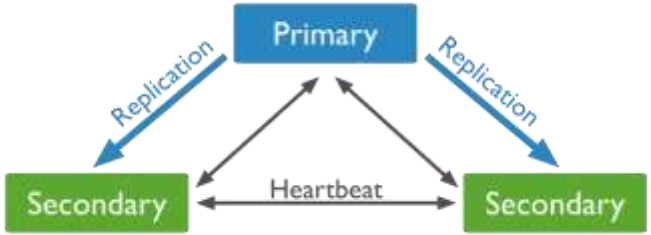
La réplication Primary-Secondary

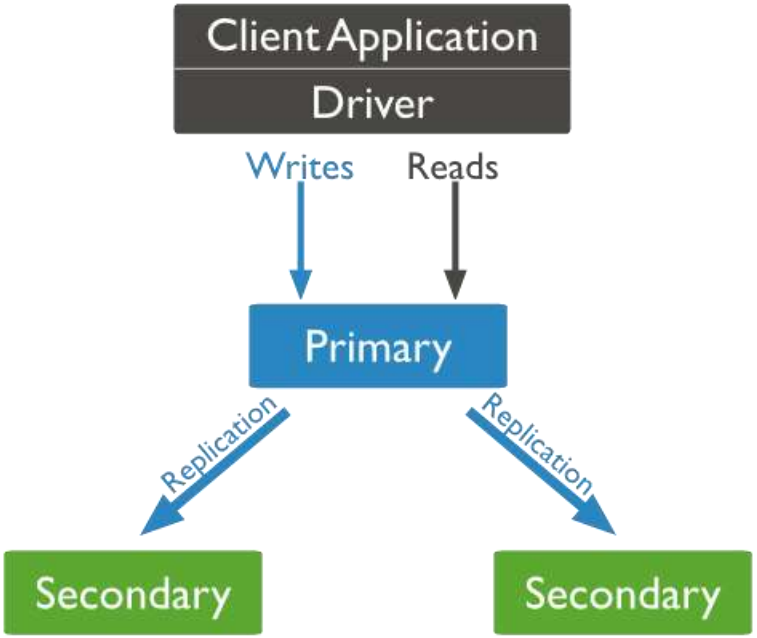
Un **replica set** est un groupe de processus mongod qui gardent les mêmes données.

Un nœud primaire
Reçoit les écritures et les note dans son journal (oplog).

Des nœuds secondaires
Copient ce journal et rejouent les opérations.

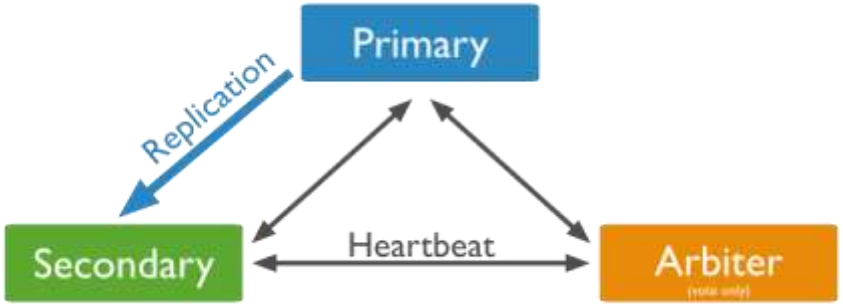
Si le primaire tombe
Un secondaire est élu primaire automatiquement.





La réplication Primary-Secondary

Un **arbitre** est une instance mongod qui vote, mais ne stocke aucune donnée.



Fiche du membre

Contient les données	✗ non
Vote aux élections	✓ oui
Peut devenir Primary	✗ non
Reçoit les lectures	✗ non

À quoi ça sert ?

Garder 3 votants à moindre coût : un Primary, un Secondary et un arbitre au lieu de deux Secondary.

Configuration

```
rs.addArb("arbitre.exemple.net:27017")
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Fonctionnement de la bascule automatique du nœud Primary

Chapitre 4

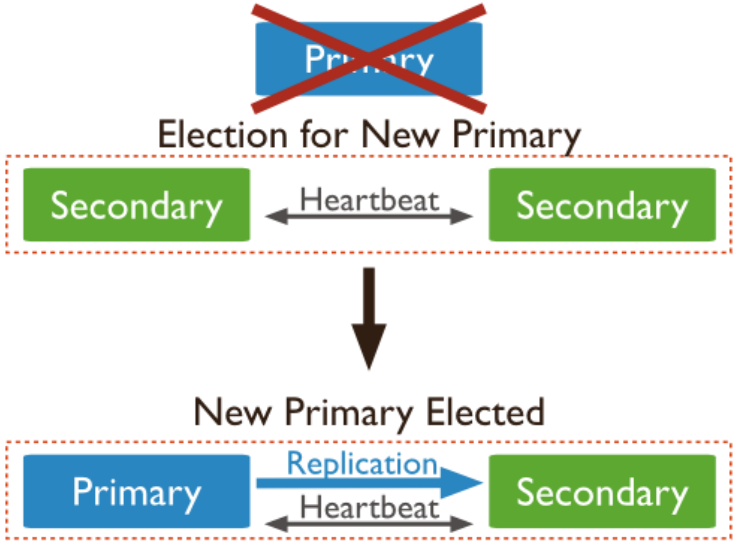
Bascule automatique du nœud Primary

Si le Primary tombe, le ReplicaSet **élit un nouveau Primary** automatiquement.

1. **Le Primary ne répond plus**
   Aucun échange avec les autres membres pendant plus de 10 secondes.

2. **Un Secondary lance l'élection**
   Il se porte candidat et demande les votes des autres membres.

3. **Le candidat élu devient Primary**
   Celui qui reçoit la majorité des votes prend la main ; les écritures reprennent.



Délai réglable : `settings.electionTimeoutMillis` (10 000 ms par défaut).

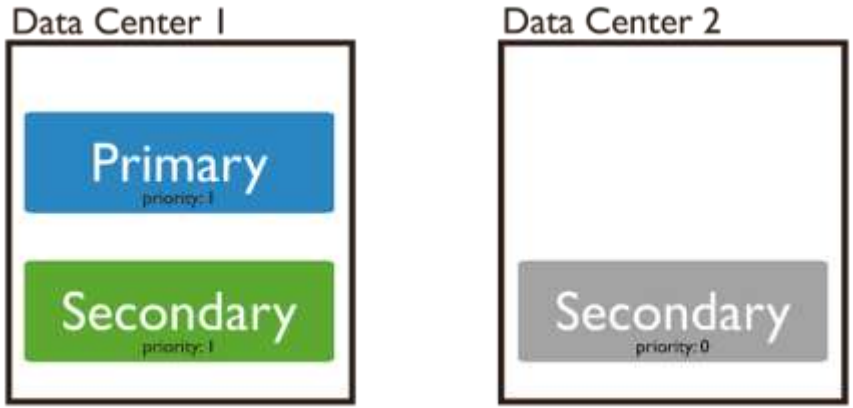


Membres secondaires particuliers

Chapitre 5

Membres à priorité nulle

Un membre de **priorité 0** ne peut jamais devenir Primary ni déclencher d'élection.



Fiche du membre

Contient les données	✓ oui
Vote aux élections	✓ oui
Peut devenir Primary	✗ non
Reçoit les lectures	✓ oui

À quoi ça sert ?

Garder un serveur de réserve dans un second datacenter, sans qu'il prenne la main.

Configuration

```
cfg = rs.conf()
cfg.members[2].priority = 0
rs.reconfig(cfg)
```

Membres cachés

Un membre **caché** (priorité 0 + hidden) est invisible pour les applications.



À quoi ça sert ?

Lancer des tâches lourdes (reporting, sauvegarde) sans ralentir les lectures des clients.

Fiche du membre

Contient les données	✓ oui
Vote aux élections	✓ oui
Peut devenir Primary	✗ non
Reçoit les lectures	✗ non

Configuration

```
cfg = rs.conf()
cfg.members[3].priority = 0
cfg.members[3].hidden = true
rs.reconfig(cfg)
```

Membres retardés

Un membre **retardé** (caché) rejoue les opérations avec un délai choisi.



Fiche du membre

Contient les données	✓ oui
Vote aux élections	✓ oui
Peut devenir Primary	✗ non
Reçoit les lectures	✗ non

À quoi ça sert ?

Revenir en arrière après une erreur humaine : la copie a encore l'état d'il y a une heure.

Configuration

```
cfg = rs.conf()
cfg.members[3].priority = 0
cfg.members[3].hidden = true
cfg.members[3].secondaryDelaySecs = 3600
rs.reconfig(cfg)
```

Membres non votants

Un membre **non votant** (votes 0, priorité 0) ne participe pas aux élections.



Fiche du membre

Contient les données	✓ oui
Vote aux élections	✗ non
Peut devenir Primary	✗ non
Reçoit les lectures	✓ oui

À quoi ça sert ?

Dépasser 7 membres : un ReplicaSet accepte jusqu'à 50 membres, mais 7 votants au maximum.

Configuration

```
cfg = rs.conf()
cfg.members[7].votes = 0
cfg.members[7].priority = 0
rs.reconfig(cfg)
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Mise en place de la réplication

Chapitre 6

Mise en place de la réplication

Six étapes pour créer un replica set nommé **`rs0`**.

1. **Démarrer chaque membre**

   en ligne de commande

   ```
   mongod --replSet "rs0"
   ```

   ou dans le fichier de configuration (YAML)

   ```
   replication:
     replSetName: "rs0"
   ```

2. **Se connecter à un membre**

   ```
   mongosh --port 27017
   ```

3. **Initier le replica set**

   ```
   rs.initiate()
   ```

4. **Vérifier la configuration initiale**

   ```
   rs.conf()
   ```

5. **Ajouter les autres membres**

   ```
   rs.add("host:port")
   ```

6. **Vérifier le statut**

   ```
   rs.status()
   ```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Read Preference

Chapitre 7

Read Preference

La Read Preference indique au driver **sur quels membres du replica set lire**.

Mode	Nœuds lus	Comportement
`primary`	**P** S S	Lit sur le primaire **(par défaut)**
`primaryPreferred`	**P** **S** **S**	Le primaire ; les secondaires s'il est indisponible
`secondary`	P **S** **S**	Lit sur les secondaires
`secondaryPreferred`	**P** **S** **S**	Les secondaires ; le primaire si aucun n'est disponible
`nearest`	P **S** S	Le nœud à la plus faible latence réseau, quel que soit son type

● lit sur ce nœud ◌ en secours P = primaire, S = secondaire

Read Preference

Lire ailleurs que sur le primaire a des usages, mais aussi des limites.

Quand changer de mode

Lectures hors temps réel
rapports, analyses

Replica set sur plusieurs datacenters
lire sur le nœud le plus proche ➔ `nearest`

Panne du primaire
garder les lectures disponibles ➔ `primaryPreferred`

Monter en charge avec `secondary` : limites

- **Même charge en écriture** sur tous les nœuds (réplication)
- **Réplication asynchrone** : un secondaire peut renvoyer des données obsolètes, deux secondaires des données différentes
- **Si un secondaire tombe**, les autres absorbent sa charge

Pour augmenter la capacité, le **sharding** est une stratégie moins contraignante.

Consistance causale

MongoDB respecte les **relations de cause à effet** entre les opérations d'une même session.

Dans une session client sur un replica set

1. Écrire
sur le primaire

→

2. Lire
sur n'importe quel membre

→

3. Résultat
l'écriture est visible

Exemple : un utilisateur modifie son profil, puis le recharge. Il voit toujours sa modification, même si la lecture part sur un secondaire.

Limite : aucune garantie vis-à-vis des **autres sessions**.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

ReadConcern

Chapitre 8

ReadConcern

`readConcern` fixe la fraîcheur des données lues, selon leur propagation dans le replica set.

`"local"` (par défaut)

Renvoie la version **la plus récente**, même si elle n'est pas encore sur une majorité de nœuds.

`"majority"`

Renvoie la version la plus récente **déjà propagée sur une majorité** de nœuds.

Requiert WiredTiger et le protocole de réplication v1.

Pourquoi majority : ne pas lire une écriture annulée (rollback)

1. Écriture sur le primaire
pas encore répliquée

2. Le primaire tombe
avant la réplication

3. Retour en secondaire
son écriture est annulée

`"local"` a pu lire cette écriture, qui n'existe plus.

`"majority"` ne l'a jamais vue : la lecture reste valide.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

WriteConcern

Chapitre 9

WriteConcern

`writeConcern` fixe le niveau d'accusé de réception attendu pour une écriture (insert, update).

```
{ w: <value>, j: <boolean>, wtimeout: <number> }
```

`w` : nombre de nœuds qui confirment l'écriture en mémoire

`w: 0`	`w: 1`	`w: n`	`w: "majority"`
aucun accusé de réception	le primaire (par défaut)	n nœuds, primaire compris	une majorité de nœuds, primaire compris

plus rapide → plus durable

`j` : écriture dans le journal

`true` : attend que l'écriture soit journalisée sur les nœuds demandés par **`w`**.

`false` par défaut, sauf avec **`w: "majority"`**.

`wtimeout` : délai maximum (ms)

S'applique si **`w > 1`**. Délai dépassé ➔ erreur renvoyée.

Sans valeur (ou 0) et niveau inatteignable ➔ **écriture bloquée indéfiniment**.



CAP COMPÉTENCES
Activateur de Talents

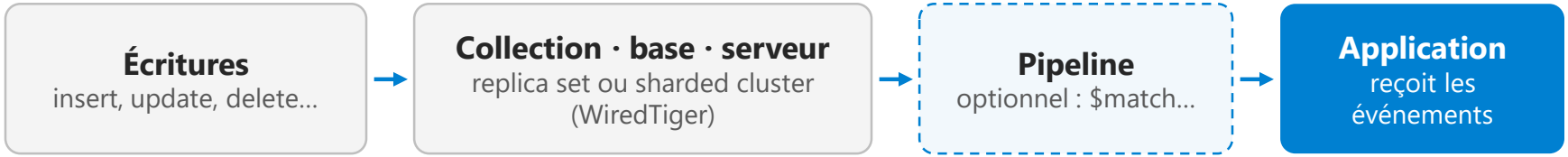
EURO INFORMATION

Flux de changement

Chapitre 10

Flux de changement

Un flux de changement (change stream) envoie à l'application **chaque modification en temps réel**.



Niveau d'écoute

Une collection, une base entière ou tout le serveur.

Filtrer et transformer

Un pipeline d'agrégation ne garde que les événements utiles.

Reprise

L'écoute peut reprendre à partir d'un point donné (resume token).

En C# écouter une collection

```
using var cursor = await collection.WatchAsync();
await cursor.ForEachAsync(change =>
    Console.WriteLine($"{change.OperationType} : {change.FullDocument}"));
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Sharding : introduction

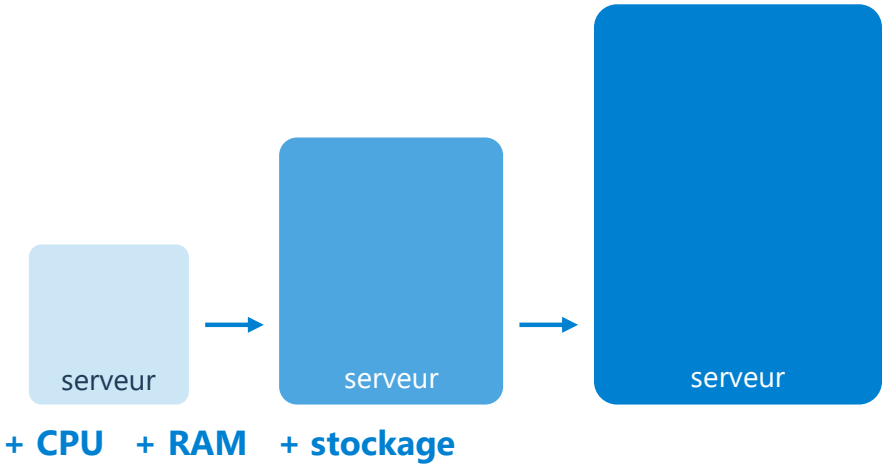
Chapitre 11

Sharding : introduction

Le sharding stocke les données d'une base **sur plusieurs machines**. Il répond aux bases dont le volume de données et de requêtes dépasse les limites d'un seul serveur.

Première option : la scalabilité verticale

ajouter des ressources au même serveur



Ses limites

Coût non proportionnel

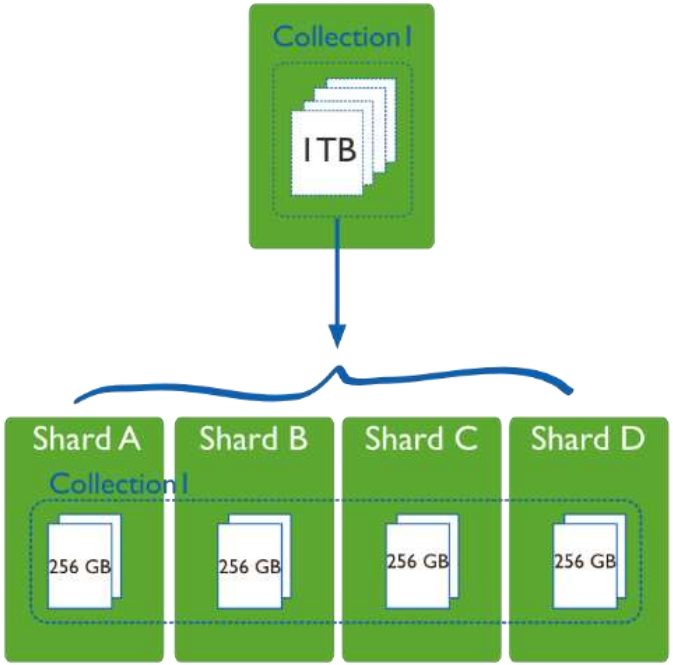
Un gros serveur coûte beaucoup plus cher que plusieurs petits.

Taille plafonnée

Les fournisseurs cloud limitent la taille des instances.

Sharding : introduction

Deuxième option : la **scalabilité horizontale**, qui répartit les données sur plusieurs serveurs appelés shards.



Données réparties

Chaque shard ne stocke qu'une partie des données.

Charge répartie

Les lectures et les écritures sont réparties entre les shards.

Une seule base vue de l'application

L'ensemble des shards forme une base logique unique.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Le sharding avec MongoDB

Chapitre 12

Le sharding avec MongoDB

Un **sharded cluster** MongoDB réunit trois types de composants.

Shards au moins 2

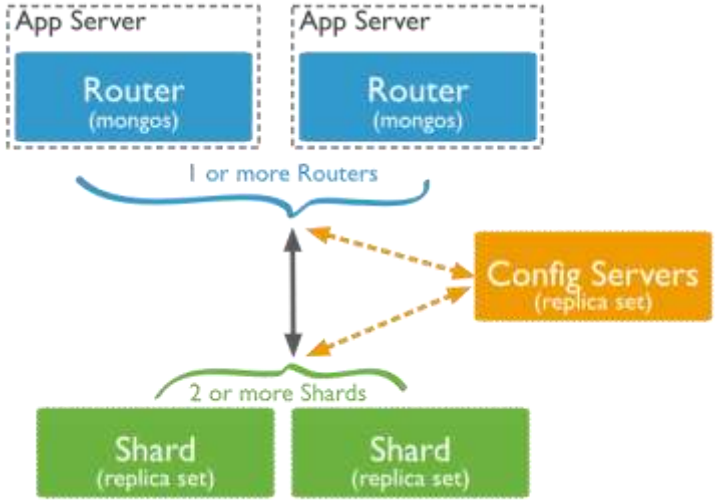
Stockent les données, chacun en replica set pour la haute disponibilité.

Routeurs mongos au moins 1

Interface entre les applications et les shards : envoient chaque requête aux bons shards et renvoient la réponse.

Config servers replica set

Stockent les métadonnées du cluster (quelle donnée sur quel shard), utilisées par les routeurs.



Le sharding avec MongoDB

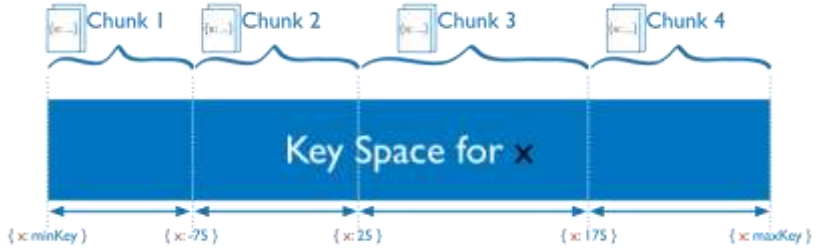
MongoDB répartit les données entre les shards **collection par collection**.

> **La clé de shard** simple ou composée
> Elle décide sur quel shard va chaque document. Idéalement, elle existe dans tous les documents de la collection.

Deux façons de répartir les valeurs de la clé

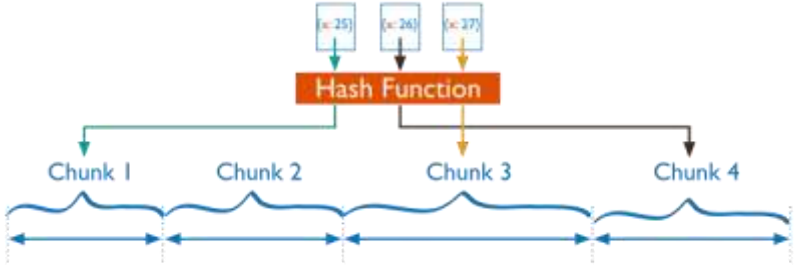
Par intervalles de valeurs

des valeurs proches vont sur le même shard



Par hash de valeurs

répartition uniforme (≥ 2.4)



Le sharding avec MongoDB

Chaque méthode de répartition a ses avantages et ses inconvénients.

	Par intervalles	**Par hash**
Requêtes sur un intervalle	✓ Efficaces : peu de shards interrogés	✗ Pas optimales : tous les shards sont interrogés
Équilibre des chunks	✗ Risque de répartition déséquilibrée des chunks, donc des requêtes	✓ Répartition plus équitable, car « aléatoire »
Clé à éviter	✗ Valeurs croissantes (date, compteur) : toutes les écritures vont sur un seul shard	✗ Peu de valeurs distinctes : limite l'efficacité de la répartition



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Mise en place du sharding

Chapitre 13

Mise en place du sharding

Les serveurs de configuration sont déployés **sous la forme d'un replica set**.

a. Démarrer les mongod en ligne de commande

```
mongod --configsvr --replSet configReplSet --port <port> --dbpath <path>
```

ou fichier de configuration

```
sharding:
  clusterRole: configsvr
replication:
  replSetName: configReplSet
```

b. Initier le replica set depuis mongosh

```
rs.initiate({
  _id: "configReplSet",
  configsvr: true,
  members: [
    { _id: 0, host: "<host1>:<port1>" },
    { _id: 1, host: "<host2>:<port2>" },
    { _id: 2, host: "<host3>:<port3>" }
  ]
})
```

Mise en place du sharding

Les routeurs **`mongos`** pointent vers les config servers ; chaque shard est un replica set.

a. Démarrer les routeurs mongos en ligne de commande

```
mongos --configdb configReplSet/<cfgsvr1:port1>,<cfgsvr2:port2>,<cfgsvr3:port3>
```

ou via la clé **`sharding.configDB`** du fichier de configuration

b. Déployer chaque shard sous la forme d'un replica set (voir Mise en place de la réplication)

c. Ajouter les shards au cluster depuis mongosh, connecté au routeur mongos

```
sh.addShard("rs1/mongodb0.example.net:27017")
sh.addShard("rs2/mongodb1.example.net:27017")
```

Mise en place du sharding

a. Activer le sharding sur la base nécessaire avant la 6.0

```
sh.enableSharding("<database>")
```

b. Créer l'index qui supporte la clé de shard

```
db.<collection>.createIndex(<shard-key-pattern>)
db.people.createIndex({ "zipcode": 1, "name": 1 })   // clé composée
db.alerts.createIndex({ "_id": "hashed" })           // ≠ index _id par défaut
```

c. Sharder la collection

```
sh.shardCollection("<database>.<collection>", <shard-key-pattern>)
sh.shardCollection("records.people", { "zipcode": 1, "name": 1 })
sh.shardCollection("events.alerts", { "_id": "hashed" })
```

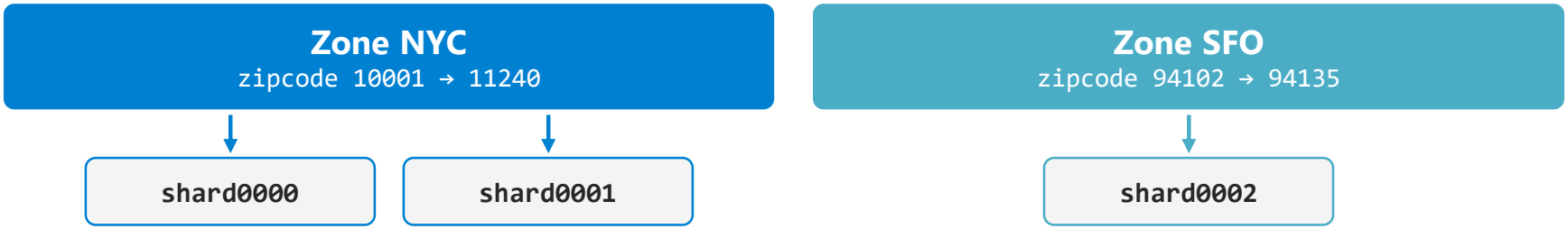


La répartition manuelle des données

- Chapitre 14

Répartition manuelle des données

Dans un cluster shardé, on peut créer des **zones** à partir de la clé de shard.



1 zone → 1 ou plusieurs shards

Chaque zone est rattachée à un ou plusieurs shards.

Pas de chevauchement

Les intervalles de deux zones ne se recouvrent jamais.

Migration automatique

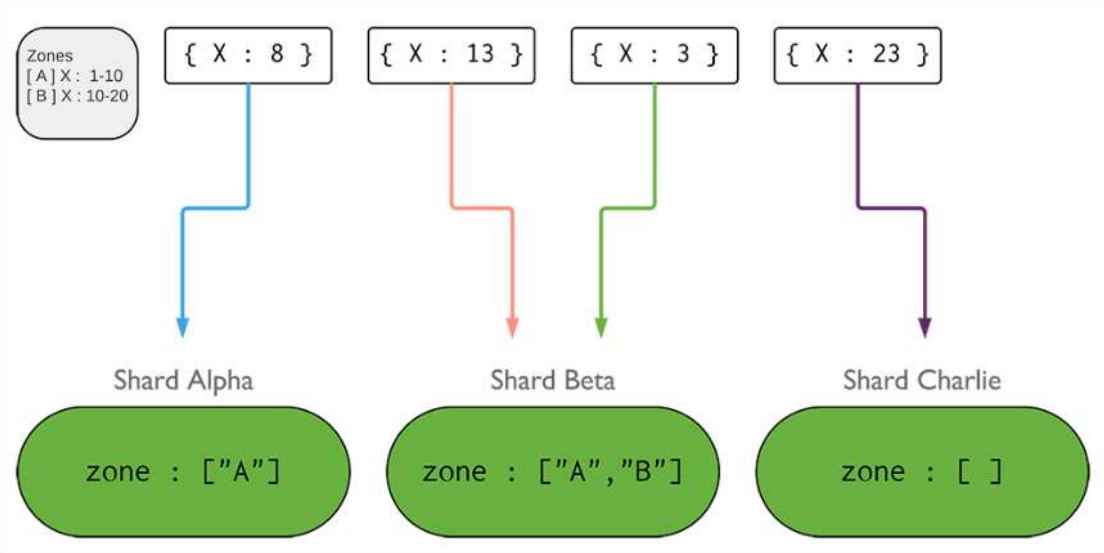
MongoDB déplace lui-même les données selon les zones.

À quoi ça sert ?

- Isoler un sous-ensemble de données sur certains shards
- Placer les données près des applications qui les lisent
- Répartir selon les performances des shards

Répartition manuelle des données

- Exemple :



- Les intervalles de valeurs sont définis limite basse incluse et limite haute exclue.

- Le « balancer » va essayer de répartir les données équitablement selon les zones cibles, ou normalement si une donnée ne rentre dans aucun intervalle.

Répartition manuelle des données

Les intervalles suivent les champs de la clé de shard, **dans l'ordre** si la clé est composée.

Exemple avec une clé { a, b, c } : chaque ligne est un découpage possible en zones.

Sur c a = 1, b = 2 fixés	`c : mini ➔ 500`	`c : 500 ➔ maxi`
Sur b a = 1 fixé	`b : mini ➔ 500`	`b : 500 ➔ maxi`
Sur a b et c libres	`a : mini ➔ 500`	`a : 500 ➔ maxi`
Sur a et b bornes composées	`{a:mini, b:mini} ➔ {a:200, b:500}`	`{a:200, b:500} ➔ {a:300, b:maxi}` …

Clé hashée seule : à éviter

Les intervalles s'écrivent sur les valeurs **avant hash**, mais les données sont réparties selon les valeurs **hashées**.

➔ **répartition imprévisible entre les zones.**

Clé mixte hashée + non hashée (≥ 4.4)

`{ "fieldA": 1, "fieldB": "hashed" }`
zones par fieldA, réparties via le hash de fieldB

`{ "_id": "hashed", "fieldA": 1 }`
hash de _id ; fieldA casse sa croissance monotone

Répartition manuelle des données

Deux instructions : **rattacher les shards aux zones**, puis **définir les intervalles**.

1 Affecter les zones aux shards

`sh.addShardTag(<shard>, <zone>)`

```
sh.addShardTag("shard0000", "NYC")
sh.addShardTag("shard0001", "NYC")
sh.addShardTag("shard0002", "SFO")
sh.addShardTag("shard0002", "NRT")
```

2 Définir les intervalles

`sh.addTagRange(<namespace>, <min>, <max>, <zone>)`

```
sh.addTagRange("records.users", { zipcode: "10001" }, { zipcode: "10281" }, "NYC")
sh.addTagRange("records.users", { zipcode: "11201" }, { zipcode: "11240" }, "NYC")
sh.addTagRange("records.users", { zipcode: "94102" }, { zipcode: "94135" }, "SFO")
```

Depuis MongoDB 3.4 : `sh.addShardToZone()` et `sh.updateZoneKeyRange()` remplacent ces deux commandes, qui restent acceptées.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

EXTENSION DE MONGODB

MODULE 6

Extension de MongoDB

- Les requêtes géo-spatiales
- L'Aggregation Framework
- Requêtes avancé avec L'Aggregation Framework



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Les requêtes géo-spatiales

Chapitre 1

Les requêtes géo-spatiales

Recherches sur des **coordonnées géographiques** (longitude, latitude) sur une sphère.

Index `2dsphere` + objets GeoJSON

`Point` `LineString` `Polygon` `MultiPoint`

`MultiLineString` `MultiPolygon` `GeometryCollection` ← mélange de tous les types

Point

```
loc: { type: "Point", coordinates: [<longitude>, <latitude>] }
```

Ligne

```
trip: { type: "LineString", coordinates: [[<longitude>, <latitude>], [<longitude>, <latitude>], …] }
```

Attention à l'ordre : la longitude vient avant la latitude.

Les requêtes géo-spatiales

Les autres formes GeoJSON : chaque niveau de crochets ajoute une dimension.

`MultiPoint` plusieurs points

```
multi_loc: { type: "MultiPoint",
  coordinates: [ [<lng>, <lat>], [<lng>, <lat>], … ] }
```

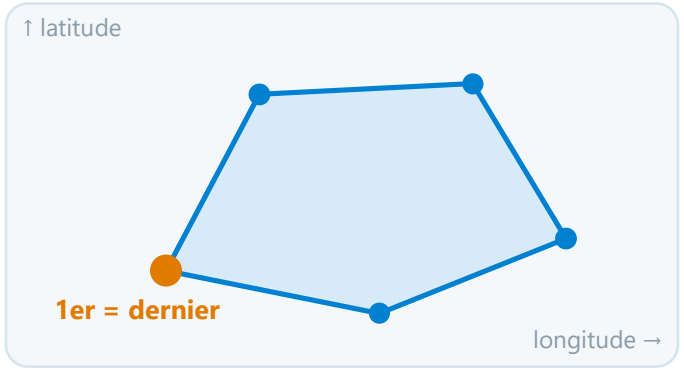
Exemple : les agences d'une même ville.



`Polygon` une zone fermée

```
shape: { type: "Polygon",
  coordinates: [
    [ [<lng>, <lat>], …, [<lng>, <lat>] ] ] }
```

Le 1er point = le dernier : l'anneau doit être fermé.

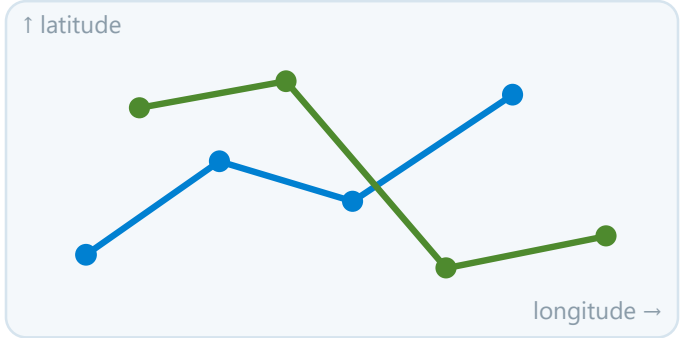


Les requêtes géo-spatiales

Les formes « Multi » regroupent **plusieurs géométries** du même type dans un seul champ.

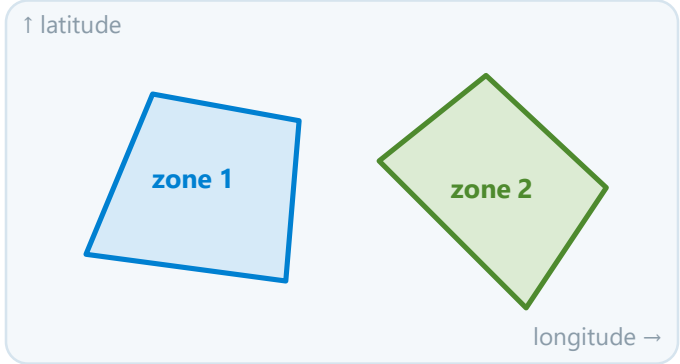
`MultiLineString` plusieurs lignes

```
multi_trip: { type: "MultiLineString",
  coordinates: [
    [ [<lng>, <lat>], [<lng>, <lat>], … ],
    [ [<lng>, <lat>], [<lng>, <lat>], … ]
  ] }
```



`MultiPolygon` plusieurs zones

```
multi_shape: { type: "MultiPolygon",
  coordinates: [
    [ [ [<lng>, <lat>], …, [<lng>, <lat>] ] ],
    [ [ [<lng>, <lat>], …, [<lng>, <lat>] ] ]
  ] }
```

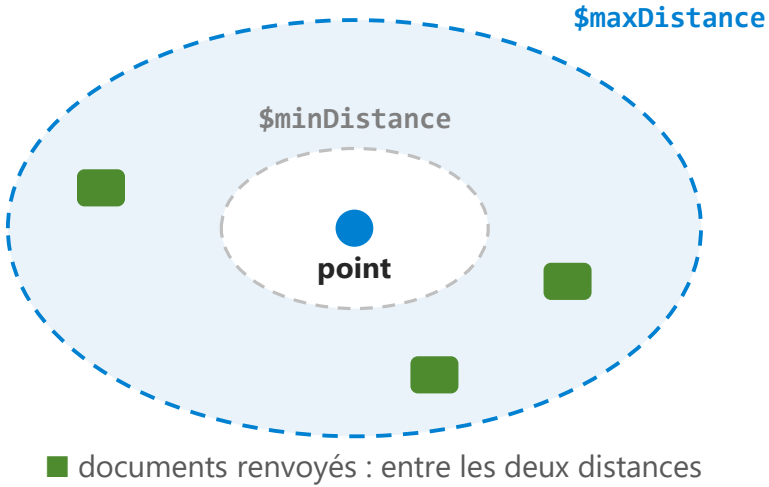


$nearSphere

$nearSphere renvoie les documents **du plus proche au plus éloigné** d'un point, sur une sphère.

Syntaxe

```
<champ GeoJSON>: {
  $nearSphere: {
    $geometry: {
      type: "Point",
      coordinates: [ <longitude>, <latitude> ]
    },
    $maxDistance: <mètres>,
    $minDistance: <mètres>
  }
}
```



documents renvoyés : entre les deux distances

Exemple agences à moins de 2 km

```
db.agences.find({ location: { $nearSphere: {
  $geometry: { type: "Point", coordinates: [ 2.3522, 48.8566 ] },
  $maxDistance: 2000 } } })
```

⚠ **Index 2dsphere requis · longitude d'abord**

$geoWithin

`$geoWithin` renvoie les documents **situés à l'intérieur d'une forme** géographique.

Formes acceptées

Polygone	Multi-polygone	Rectangle	Cercle
`Polygon`	`MultiPolygon`	`$box`	`$center` · `$centerSphere`

Syntaxe polygone GeoJSON

```
<champ GeoJSON>: {
  $geoWithin: {
    $geometry: {
      type: "Polygon",    // ou "MultiPolygon"
      coordinates: [ <coordonnées> ]
    }
  }
}
```

Exemple lieux dans une zone

```
db.places.find({
  location: { $geoWithin: {
    $geometry: { type: "Polygon",
      coordinates: [[ [2.29, 48.85],
        [2.36, 48.85], [2.36, 48.88],
        [2.29, 48.88], [2.29, 48.85] ]] }
  } }
})
```

Un polygone se referme : le dernier point est identique au premier.

$geoIntersects

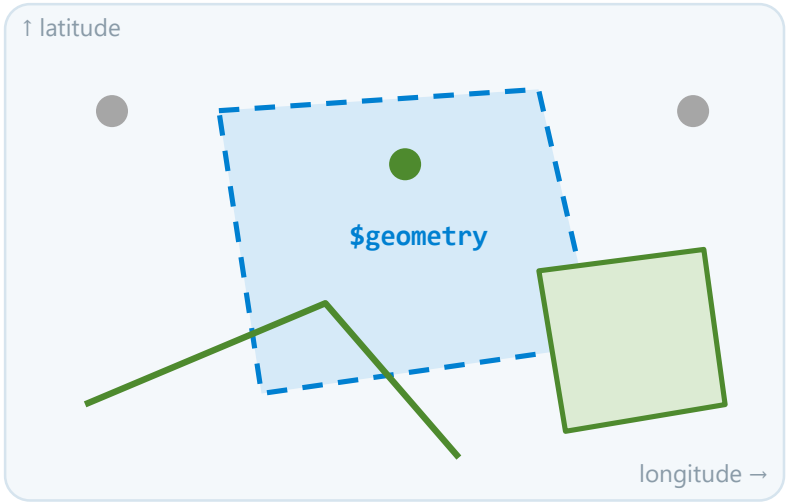
`$geoIntersects` renvoie les documents dont l'objet GeoJSON **croise ou touche** une forme donnée.

Syntaxe avec un polygone

```
<champ GeoJSON>: {
  $geoIntersects: {
    $geometry: {
      type: "Polygon",
      coordinates: [ <coordonnées> ]
    }
  }
}
```

Exemple trajets qui traversent une zone

```
db.trajets.find({ trip: {
  $geoIntersects: { $geometry: zoneParis }
} })
```



- croise ou touche la zone : renvoyé
- entièrement en dehors : ignoré

Mongofiles

`mongofiles` manipule les fichiers stockés dans **GridFS**.

Syntaxe

```
mongofiles <options> <commande> <fichier>
```

Élément	Rôle
`options`	contrôlent le comportement du programme
`commande`	l'action souhaitée
`fichier`	nom du fichier sur disque ou dans GridFS

Options

Option	Rôle
`--host <hostname>`	l'hôte
`--port <port>`	le port
`--db <database>`	la base
`--type <mime>`	type du fichier (action put)
`--replace`	remplace le fichier de même nom au lieu d'en ajouter un
`--prefix <string>`	autre préfixe que « fs »
`--local <fileName>`	nom sur disque différent du nom dans GridFS

Mongofiles

Les commandes disponibles, puis des exemples sur la base `records`.

Commande	Rôle
`list <prefix>`	liste les fichiers (commençant éventuellement par le préfixe)
`search <string>`	liste les fichiers dont le nom contient la chaîne
`put <filename>`	copie un fichier vers GridFS
`get <filename>`	récupère un fichier depuis GridFS
`delete <filename>`	supprime un fichier de GridFS
`get_id "<ObjectId>"`	récupère un fichier par son identifiant
`delete_id "<ObjectId>"`	supprime un fichier par son identifiant

Exemples

```
mongofiles -d records list                # liste les fichiers
mongofiles -d records search corinth      # cherche des fichiers
mongofiles -d records put 32-corinth.lp   # stocke un fichier
mongofiles -d records get 32-corinth.lp   # récupère un fichier
mongofiles -d records delete 32-corinth.lp  # supprime un fichier
```



CAP COMPÉTENCES
Activateur de Talents

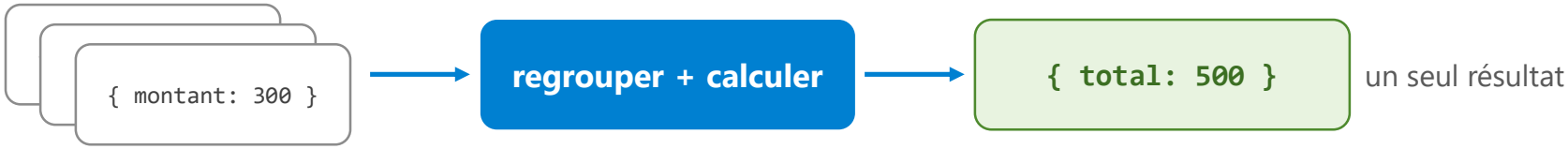
EURO INFORMATION

L'Aggregation Framework

Chapitre 2

L'Aggregation Framework

L'agrégation traite **plusieurs documents** pour renvoyer **un résultat calculé**.



Trois méthodes d'agrégation

Pipeline d'agrégation

Recommandé

Une suite d'étapes : $match, $group, $sort…

```
aggregate([ … ])
```

MapReduce

Déprécié depuis 5.0

Fonctions JavaScript map et reduce.

```
mapReduce(map, reduce)
```

Méthodes simples

Usage ponctuel

Un seul calcul, sur une collection.

```
countDocuments() · distinct()
```

L'Aggregation Framework

Le pipeline d'agrégation est une **suite d'étapes** que les documents traversent pour être agrégés.



Autres étapes possibles : transformations ($project, $unwind), jointures ($lookup), calculs statistiques…

Opérateurs natifs

Agrégation efficace, exécutée par le serveur : c'est la méthode recommandée dans MongoDB.

Index utilisés

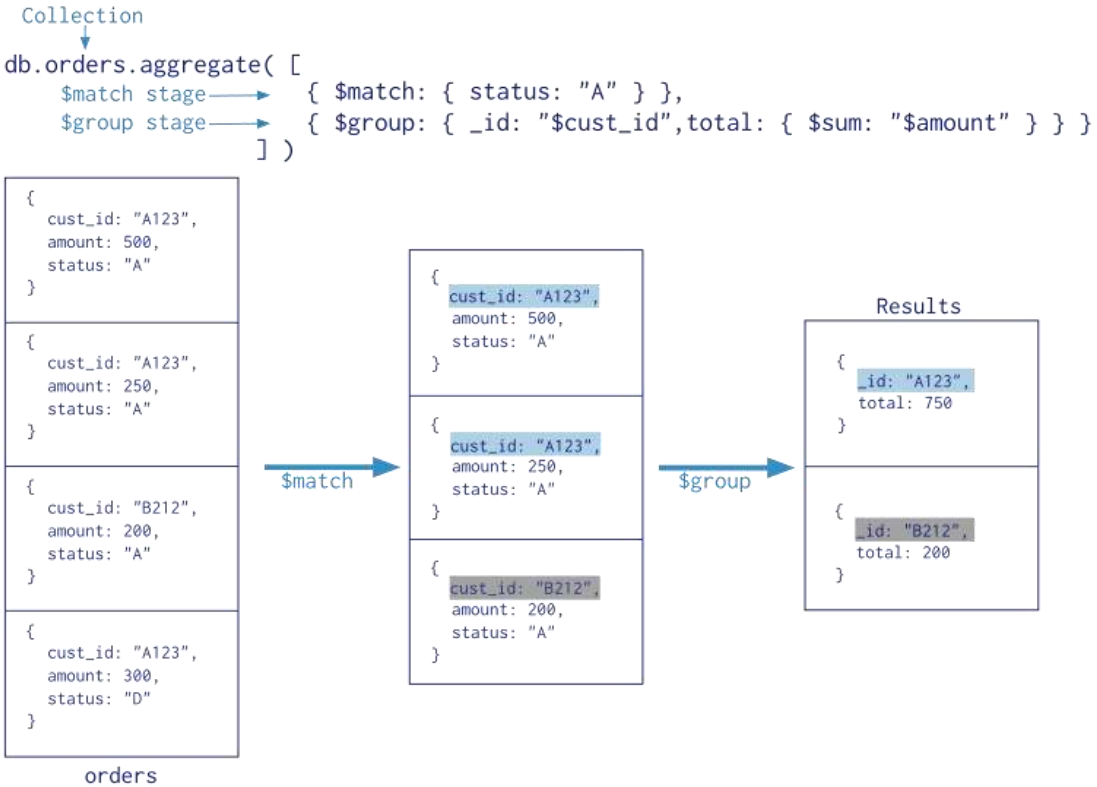
Certaines étapes, comme un $match ou un $sort en tête de pipeline, profitent des index.

Optimisation interne

MongoDB peut réordonner ou fusionner des étapes avant d'exécuter le pipeline.

L'Aggregation Framework

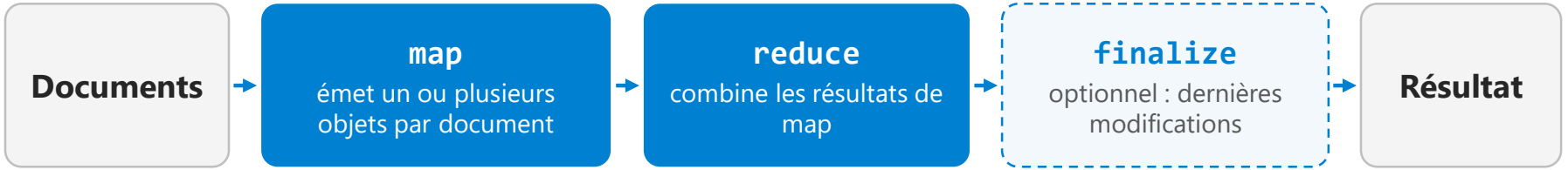
- Exemple de pipeline d'agrégation :



L'Aggregation Framework

MapReduce agrège des données avec des fonctions JavaScript.

Déprécié en 5.0



Ce qu'il permet

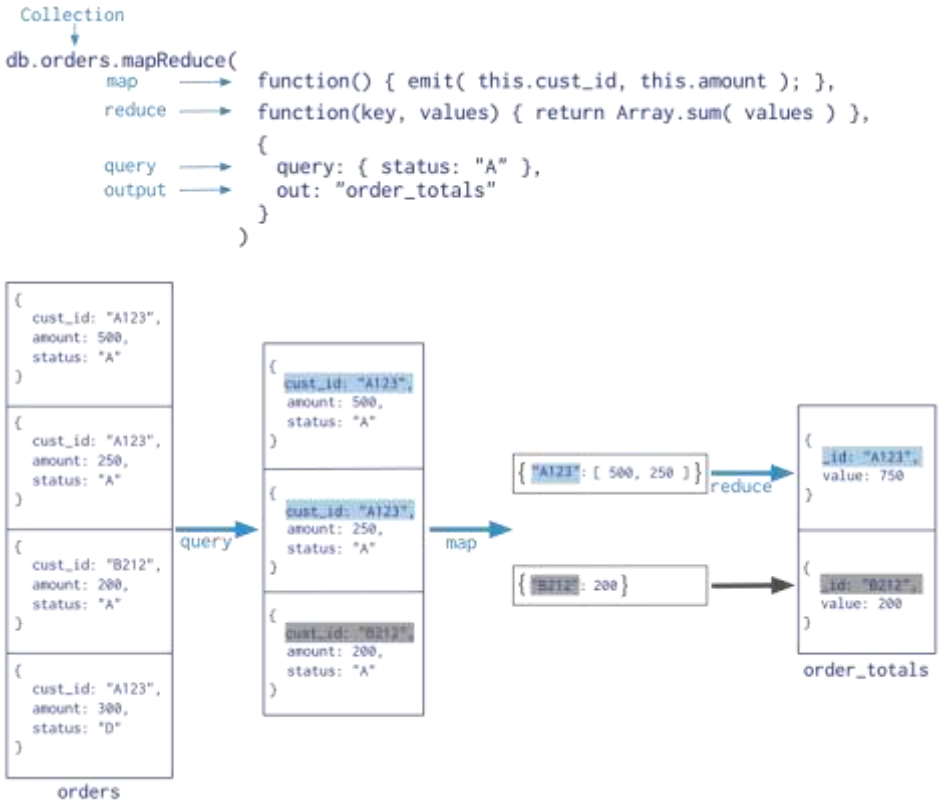
- **Plus de flexibilité** grâce au JavaScript
- **Agrégation incrémentale** : n'ajouter que les nouveaux documents

Ses limites

- **Moins efficace** que le pipeline d'agrégation
- **Déprécié en 5.0** : préférer le pipeline pour tout nouveau code

L'Aggregation Framework

- Exemple de MapReduce :



L'Aggregation Framework

Pour les cas simples, MongoDB fournit **deux fonctions d'agrégation** sans pipeline.

Compter

les documents qui correspondent à un filtre

```
db.collection.countDocuments(query, options)
```

`query`	filtre de recherche
`options`	document d'options

Exemple

```
db.clients.countDocuments({ City: "Paris" })
```

Valeurs distinctes

d'un champ, sans doublon

```
db.collection.distinct(field, query, options)
```

`field`	champ à retourner
`query`	filtre de recherche
`options`	document d'options

Exemple

```
db.clients.distinct("City")
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Requêtes avancées et agrégations

Module 7

Requêtes avancées et Agrégation

- Maîtriser les principales étapes (`$match`, `$group`, `$project`, `$lookup`, `etc.`).
- Utiliser l'API C# Aggregate() pour construire des requêtes complexes.
- Comparer l'agrégation MongoDB à une requête SQL équivalente.



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Maîtriser les principales étapes de l'Agrégation

Chapitre 1

Étape $match – Filtrer

- $match garde les documents qui respectent le filtre. À placer en tête : il utilise les index.

Pipeline mongosh

```
db.transactions.aggregate([
  { $match: {
      Type: "Virement",
      Amount: { $gte: 1000 },
      Date: { $gte: ISODate("2025-01-01") }
  } }
])
```

Résultat exemple de sortie

```
{ ClientId: "C001", Type: "Virement",
  Amount: 2500, Status: "Validée",
  Date: ISODate("2025-03-12") }
{ ClientId: "C007", Type: "Virement",
  Amount: 1200, Status: "Rejetée",
  Date: ISODate("2025-04-02") }
```

Équivalent SQL

```
SELECT * FROM transactions
WHERE Type = 'Virement' AND Amount >= 1000 AND Date >= '2025-01-01';
```

Même syntaxe de filtre que find() : $gte, $in, $or... fonctionnent à l'identique.

Étape $project / $sort / $limit – Mettre en forme

- $project choisit et calcule les champs, $sort trie (1 / -1), $limit coupe le résultat.

Pipeline mongosh

```
db.transactions.aggregate([
  { $match: { Type: "Retrait" } },
  { $project: {
      _id: 0, ClientId: 1, Amount: 1, Date: 1,
      Mois: { $month: "$Date" }      // champ calculé
  } },
  { $sort: { Amount: -1 } },         // -1 = décroissant
  { $limit: 3 }
])
```

Résultat exemple de sortie

```
{ ClientId: "C003", Amount: 4000,
  Date: ISODate("2025-05-20"), Mois: 5 }
{ ClientId: "C001", Amount: 1800,
  Date: ISODate("2025-02-14"), Mois: 2 }
{ ClientId: "C007", Amount: 950,
  Date: ISODate("2025-06-01"), Mois: 6 }
```

Équivalent SQL

```
SELECT ClientId, Amount, Date, MONTH(Date) AS Mois FROM transactions
WHERE Type = 'Retrait' ORDER BY Amount DESC LIMIT 3;
```

Le préfixe $ ("$Date") désigne la valeur d'un champ du document en cours.

Étape $group – Regrouper et calculer

- $group crée un document par valeur de _id et calcule des agrégats : $sum, $avg, $max…

Pipeline mongosh

```
db.transactions.aggregate([
  { $match: { Status: "Validée" } },
  { $group: {
      _id: "$ClientId",              // clé de regroupement
      Total: { $sum: "$Amount" },
      Moyenne: { $avg: "$Amount" },
      Nb: { $sum: 1 }                // compte les documents
  } },
  { $sort: { Total: -1 } }
])
```

Résultat exemple de sortie

```
{ _id: "C001", Total: 6300,
  Moyenne: 2100, Nb: 3 }
{ _id: "C003", Total: 4000,
  Moyenne: 4000, Nb: 1 }
{ _id: "C007", Total: 2150,
  Moyenne: 1075, Nb: 2 }
```

Équivalent SQL

```
SELECT ClientId, SUM(Amount) AS Total, AVG(Amount) AS Moyenne, COUNT(*) AS Nb
FROM transactions WHERE Status = 'Validée' GROUP BY ClientId ORDER BY Total DESC;
```

_id: null regroupe toute la collection en un seul document (total général).

Étape $unwind – Déplier un tableau

- $unwind crée un document par élément d'un tableau, pour pouvoir le filtrer ou le regrouper.

Pipeline mongosh

```
// { _id: "C001", Name: "Alice Martin",
//   Accounts: [ { Type: "Courant", Balance: 2500 },
//               { Type: "Épargne", Balance: 8000 } ] }
db.clients.aggregate([
  { $unwind: "$Accounts" },          // 1 doc par compte
  { $group: {
      _id: "$Accounts.Type",
      Encours: { $sum: "$Accounts.Balance" }
  } }
])
```

Résultat après $unwind, puis $group

```
// après $unwind
{ _id: "C001",
  Accounts: { Type: "Courant",
              Balance: 2500 } }
{ _id: "C001",
  Accounts: { Type: "Épargne",
              Balance: 8000 } }
// après $group
{ _id: "Courant", Encours: 14200 }
{ _id: "Épargne", Encours: 31500 }
```

Équivalent SQL

```
SELECT a.Type, SUM(a.Balance) AS Encours
FROM clients c JOIN accounts a ON a.ClientId = c._id GROUP BY a.Type;
```

preserveNullAndEmptyArrays: true garde aussi les clients sans compte.

Étape $lookup – Joindre deux collections

- $lookup ajoute un tableau des documents liés ; $unwind le transforme en un objet par ligne.

Pipeline mongosh

```
db.transactions.aggregate([
  { $lookup: {
      from: "clients",           // collection jointe
      localField: "ClientId",    // champ de transactions
      foreignField: "_id",       // champ de clients
      as: "Client"               // tableau ajouté
  } },
  { $unwind: "$Client" },        // tableau → objet
  { $project: { _id: 0, Amount: 1, Nom: "$Client.Name" } }
])
```

Résultat exemple de sortie

```
{ Amount: 2500, Nom: "Martin Dupont" }
{ Amount: 1200, Nom: " Anas Ben Yaiche" }
{ Amount: 1800, Nom: " Martin Dupont" }
```

Équivalent SQL

```
SELECT t.Amount, c.Name AS Nom
FROM transactions t LEFT JOIN clients c ON t.ClientId = c._id;
```

Sans $unwind, Client reste un tableau (vide si aucun client ne correspond).

Étape $graphLookup – Parcourir une hiérarchie

- $graphLookup suit un lien de proche en proche : ici, d'un sous-compte jusqu'au compte racine.

Pipeline mongosh

```
db.accounts.aggregate([
  { $match: { AccountNumber: "ACC-300" } },
  { $graphLookup: {
      from: "accounts",
      startWith: "$ParentAccount",        // point de départ
      connectFromField: "ParentAccount", // lien à suivre
      connectToField: "AccountNumber",
      as: "Hierarchy",
      maxDepth: 5, depthField: "Niveau"
  } }
])
```

Résultat exemple de sortie

```
{ AccountNumber: "ACC-300",
  ParentAccount: "ACC-200",
  Hierarchy: [
    { AccountNumber: "ACC-200",
      ParentAccount: "ACC-100",
      Niveau: 0 },
    { AccountNumber: "ACC-100",
      ParentAccount: null, Niveau: 1 }
  ] }
```

Équivalent SQL

```
WITH RECURSIVE h AS (SELECT * FROM accounts WHERE AccountNumber = 'ACC-200'
  UNION ALL SELECT a.* FROM accounts a JOIN h ON a.AccountNumber = h.ParentAccount) SELECT * FROM h;
```

Étape $out / $merge – Enregistrer le résultat

- Toujours en dernière étape. $out remplace la collection cible, $merge la met à jour.

$out remplace la collection

```
db.transactions.aggregate([
  { $match: { Status: "Validée" } },
  { $out: "transactions_validees" }
])
```

$merge insère ou met à jour

```
db.transactions.aggregate([
  { $group: { _id: "$ClientId",
              Solde: { $sum: "$Amount" } } },
  { $merge: { into: "soldes",
              whenMatched: "replace",
              whenNotMatched: "insert" } }
])
```

Équivalent SQL

```
CREATE TABLE transactions_validees AS SELECT * FROM transactions WHERE Status = 'Validée';  -- $out
MERGE INTO soldes ...                                                                       -- $merge
```

Idéal pour précalculer des rapports (soldes, totaux mensuels) lus ensuite par find().



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

l'Agrégation avec C#

Chapitre 2

Agrégation avec C# – Modèle

Deux collections, **transactions** et **clients**, reliées par ClientId (champs en PascalCase).

Transaction collection « transactions »

```
public class Transaction
{
    public ObjectId Id { get; set; }
    public string ClientId { get; set; }
    public string Type { get; set; }    // Virement | Retrait
    [BsonRepresentation(BsonType.Decimal128)]
    public decimal Amount { get; set; }
    public DateTime Date { get; set; }
    public string Status { get; set; }  // Validée | Rejetée
}
```

Client collection « clients »

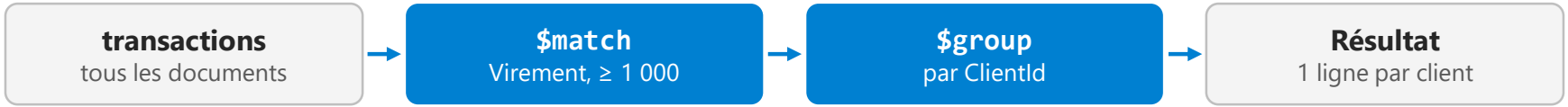
```
public class Client
{
    [BsonId]  // ex. "C001"
    public string Id { get; set; }
    public string Name { get; set; }
    public string City { get; set; }
    public string Segment { get; set; }
}
```

Collections using MongoDB.Driver · MongoDB.Driver.Linq

```
var transactionsDoc = db.GetCollection<BsonDocument>("transactions"); // BsonDocument
var transactions    = db.GetCollection<Transaction>("transactions");  // Aggregate() et LINQ
var clients         = db.GetCollection<Client>("clients");
```

Agrégation avec C# – $match / $group (1/2)

Total et nombre de **virements ≥ 1 000** par client : on filtre, puis on regroupe.



BsonDocument

le pipeline écrit comme dans le shell

```
var match = new BsonDocument("$match", new BsonDocument {
    { "Type", "Virement" }, { "Amount", new BsonDocument("$gte", 1000) } });
var group = new BsonDocument("$group", new BsonDocument {
    { "_id", "$ClientId" },
    { "Total", new BsonDocument("$sum", "$Amount") },
    { "Nb", new BsonDocument("$sum", 1) } });
var result = await transactionsDoc
    .Aggregate<BsonDocument>(new[] { match, group }).ToListAsync();
```

Résultat `{ _id: "C001", Total: 4500, Nb: 3 }` · `{ _id: "C002", Total: 1200, Nb: 1 }`

Agrégation avec C# – $match / $group (2/2)

Le même pipeline, **typé** : le compilateur vérifie les noms de champs.

Aggregate() + POCO

API fluide typée

```
var result = await transactions.Aggregate()
    .Match(t => t.Type == "Virement" && t.Amount >= 1000)
    .Group(t => t.ClientId,
           g => new { ClientId = g.Key, Total = g.Sum(t => t.Amount), Nb = g.Count() })
    .ToListAsync();
```

LINQ

AsQueryable(), traduit en $match + $group

```
var result = await transactions.AsQueryable()
    .Where(t => t.Type == "Virement" && t.Amount >= 1000)
    .GroupBy(t => t.ClientId)
    .Select(g => new { ClientId = g.Key, Total = g.Sum(t => t.Amount), Nb = g.Count() })
    .ToListAsync();
```

Les trois versions **envoient le même pipeline** au serveur : le choix dépend du typage voulu.

Agrégation avec C# – $sort / $limit / $project (1/2)

Les **5 plus gros retraits** depuis le 1er janvier 2025, du plus élevé au plus faible.

$match
Retrait, depuis 2025
→ **$sort**
Amount décroissant
→ **$limit**
5 premiers
→ **$project**
3 champs gardés

BsonDocument le pipeline écrit comme dans le shell

```
var since = new DateTime(2025, 1, 1, 0, 0, 0, DateTimeKind.Utc);
var pipeline = new BsonDocument[] {
    new("$match", new BsonDocument {
        { "Type", "Retrait" }, { "Date", new BsonDocument("$gte", since) } }),
    new("$sort", new BsonDocument("Amount", -1)),
    new("$limit", 5),
    new("$project", new BsonDocument {
        { "_id", 0 }, { "ClientId", 1 }, { "Amount", 1 }, { "Date", 1 } }) };
var top5 = await transactionsDoc.Aggregate<BsonDocument>(pipeline).ToListAsync();
```

L'ordre compte : `$sort` **avant** `$limit`, sinon on garde 5 retraits au hasard, puis on les trie.

Agrégation avec C# – $sort / $limit / $project (2/2)

En typé, chaque étape a sa méthode : **`SortByDescending`** ou **`OrderByDescending`**.

Aggregate() + POCO — API fluide typée

```
var top5 = await transactions.Aggregate()
    .Match(t => t.Type == "Retrait" && t.Date >= since)
    .SortByDescending(t => t.Amount)
    .Limit(5)
    .Project(t => new { t.ClientId, t.Amount, t.Date }).ToListAsync();
```

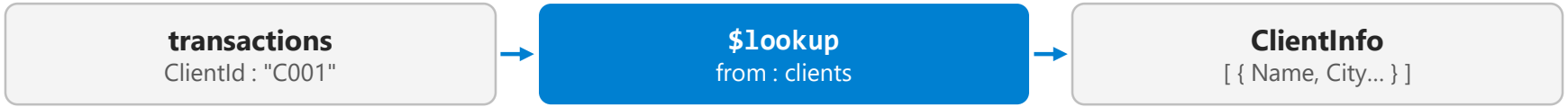
LINQ — AsQueryable(), traduit en pipeline

```
var top5 = await transactions.AsQueryable()
    .Where(t => t.Type == "Retrait" && t.Date >= since)
    .OrderByDescending(t => t.Amount)
    .Take(5)
    .Select(t => new { t.ClientId, t.Amount, t.Date }).ToListAsync();
```

Correspondances LINQ : Where → $match · OrderBy → $sort · Take → $limit · Select → $project

Agrégation avec C# – $lookup (1/2)

$lookup ajoute à chaque transaction **le client correspondant** (ClientId → clients._id).



BsonDocument — le pipeline écrit comme dans le shell

```
var lookup = new BsonDocument("$lookup", new BsonDocument
{
    { "from", "clients" },          { "localField", "ClientId" },
    { "foreignField", "_id" },      { "as", "ClientInfo" }
});
var result = await transactionsDoc
    .Aggregate<BsonDocument>(new[] { lookup }).ToListAsync();
```

Résultat

```
{ ClientId: "C001", Amount: 1500,
            ClientInfo: [ { _id: "C001", Name: "Anas Ben Yaiche", City: "Paris" } ] }
```

Agrégation avec C# – $lookup (2/2)

En typé, on décrit le résultat avec une classe, ou on **joint en LINQ**.

Aggregate() + POCO une classe pour le résultat

```
public class TransactionAvecClient : Transaction { public Client[] ClientInfo { get; set; } }

var result = await transactions.Aggregate()
    .Lookup<Client, TransactionAvecClient>(clients, t => t.ClientId, c => c.Id, r => r.ClientInfo)
    .ToListAsync();
```

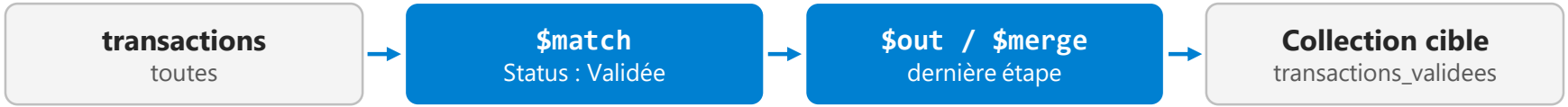
LINQ Join(), traduit en $lookup + $unwind

```
var result = await transactions.AsQueryable()
    .Join(clients.AsQueryable(),
          t => t.ClientId, c => c.Id,
          (t, c) => new { t.Amount, t.Date, Client = c.Name })
    .ToListAsync();
```

Différence : Lookup() renvoie un tableau ClientInfo ; Join() renvoie une ligne par paire.

Agrégation avec C# – $out / $merge

`$out` remplace une collection par le résultat ; **`$merge`** la **met à jour**.



BsonDocument

$out ou $merge en dernière étape

```
var pipeline = new BsonDocument[] {
    new("$match", new BsonDocument("Status", "Validée")),
    new("$out", "transactions_validees")
    // ou : new("$merge", new BsonDocument { { "into", "soldes" }, { "whenMatched", "replace" } })
};
await transactionsDoc.Aggregate<BsonDocument>(pipeline).ToListAsync();
```

Aggregate() + POCO

API fluide typée

```
await transactions.Aggregate()
    .Match(t => t.Status == "Validée")
    .OutAsync("transactions_validees");
```

LINQ : non applicable

AsQueryable() ne fait que lire : on passe par Aggregate() pour écrire.

Agrégation avec C# – Quelle approche choisir ?

- Les trois approches produisent le même pipeline ; elles diffèrent par le typage.

	BsonDocument	Aggregate() + POCO	LINQ
Typage	Aucun (chaînes)	Fort (lambdas)	Fort (lambdas)
Faute sur un nom de champ	Vue à l'exécution	Vue à la compilation	Vue à la compilation
Étapes disponibles	Toutes	Toutes (AppendStage)	Where, GroupBy, Join…
$out / $merge	Oui	Oui	Non
Usage conseillé	Pipeline copié du shell	Pipelines métier	Lectures simples



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Comparaison entre l'Agrégation MongoDB et SQL

Chapitre 3

Comparatif SQL vs C# Aggregation

Chaque clause SQL a son **étape équivalente** dans le pipeline d'agrégation.

	SQL (relationnel)	MongoDB · C#
Principe	Clauses d'une requête	Pipeline d'étapes successives
Langage	Déclaratif	Typé en C# (Aggregate() ou LINQ)
Filtrage	`WHERE · HAVING`	`$match`
Regroupement	`GROUP BY`	`$group`
Tri · limite	`ORDER BY · LIMIT`	`$sort · $limit`
Fonctions	`SUM AVG COUNT MAX MIN`	`$sum $avg $count $max $min`
Résultat	Lignes d'un tableau	Documents BSON (objets C#)

Exemple total des virements validés par client

```
SELECT ClientId, SUM(Amount)
FROM transactions
WHERE Status = 'Validée'
GROUP BY ClientId
```

→

```
transactions.Aggregate()
  .Match(t => t.Status == "Validée")
  .Group(t => t.ClientId,
    g => new { g.Key, Total = g.Sum(t => t.Amount) })
```



CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION

Conclusion

Conclusion

Ce que vous avez appris

1. **NoSQL**
   La philosophie NoSQL et le modèle document
2. **MongoDB**
   Installer, configurer et manipuler MongoDB
3. **CRUD**
   Opérations via le shell et le driver C#
4. **Modélisation**
   Validation et indexation des données
5. **Performance**
   Optimiser avec explain() et le profiler

Ce que vous savez faire

- ✓ **Concevoir**
  une application C# connectée à MongoDB
- ✓ **Structurer**
  vos modèles en imbriqué ou en référence
- ✓ **Gérer**
  performance et montée en charge : réplication, sharding, agrégation
- ✓ **Mettre en place**
  une architecture robuste et extensible

Vrai ou faux ? Les réponses

Retour sur les affirmations du début du cours.

« Sans schéma, pas besoin de modéliser ses données. »

Le schéma est flexible, mais le choix **imbriquer ou référencer** décide des performances.

✗ **FAUX**

« MongoDB ne gère pas les transactions. »

Transactions **multi-documents** depuis la 4.0 (replica set) et la 4.2 (sharded cluster).

✗ **FAUX**

« Un document MongoDB ne peut pas dépasser 16 Mo. »

Limite d'un document BSON : **16 Mo**. Au-delà, on stocke les fichiers avec GridFS.

✓ **VRAI**

Pour terminer

Quelques questions pour faire le point sur la formation.

Testez-vous

1. Quand imbriquer un document plutôt que le référencer ?
2. Quelle approche C# choisir : BsonDocument, Builders ou LINQ ?
3. Comment vérifier qu'une requête utilise bien un index ?

Tour de table

? Qu'allez-vous appliquer dès demain dans vos projets ?

? Quel sujet voulez-vous approfondir ?

? Vos questions ouvertes

Merci !

CAP COMPÉTENCES
Activateur de Talents

EURO INFORMATION